

ML Optimized Stochastic Vehicle Routing for Urban Delivery System

A PROJECT REPORT

Submitted by

NAME OF THE CANDIDATE(S)

Aryan Dhasmana(23BCB10081)

Nikhil Gusain (23BCB10033)

Simranjit kaur (23BCB10084)

Isha Agarwal(23BCB10053)

Vishal Gupta (23BCB10080)

in partial fulfillment for the award of the degree of

Bachelor of Engineering

IN

COMPUTER SCIENCE ENGINEERING (CSBS)



April 2026



BONAFIDE CERTIFICATE

Certified that this project report “**ML-Optimized Stochastic Vehicle Routing for Urban Delivery Services**” is the bonafide work of “**Aryan Dhasmana, Simranjit Kaur, Nikhil Gusain, Isha Aggarwal, Vishal Gupta**” who carried out the project work under my/our supervision.

SIGNATURE

Dr .Aman Kaushik

HEAD OF THE DEPARTMENT

AIT-CSE, Chandigarh University

SIGNATURE

Dr. Gurwinder Singh

SUPERVISOR

Assistant Professor

Department of CSE, Chandigarh
University

Submitted for the project viva-voce examination held on 30th April,2026

ACKNOWLEDGEMENT

This project report is the outcome of sustained academic effort, collaborative teamwork, and the institutional support made available during the undergraduate project cycle at Chandigarh University. We express our sincere gratitude to our supervisor, Dr. Gurwinder Singh, Assistant Professor, AIT-CSE, for his expert guidance, patient review of the evolving benchmark design, and continuous encouragement during every phase of the project. His observations helped us convert a broad interest in delivery-route optimisation into a disciplined study grounded in reproducibility, solver comparison, and engineering clarity.

We also convey our heartfelt thanks to Dr. Aman Kaushik, Head of the Department, for providing the academic environment, administrative support, and institutional encouragement required to pursue a project of this scale. His support strengthened the connection between the project’s technical objectives and the broader goals of the Computer Science and Business Systems programme, especially the need to combine algorithmic rigour with business relevance and social impact.

We gratefully acknowledge Chandigarh University for providing the facilities, laboratory access, software ecosystem, and computing resources that made the work possible. In particular, the university environment allowed us to conceptualise an OSMnx-accessible urban-routing workflow, build a reproducible stochastic benchmark scaffold, evaluate multiple solvers under controlled simulation, and prepare this report with the professionalism expected of an official undergraduate submission.

We would also like to thank every member of our project team. The report reflects a genuinely collaborative effort in which data design, solver implementation, evaluation planning, visual reporting, writing, and revision were shared responsibly. The willingness of team members to review one another’s work, validate assumptions, and maintain consistency across technical sections significantly improved the quality of the final outcome.

Finally, we acknowledge our families and friends for their encouragement, patience, and confidence throughout the project period. Their support helped us remain committed during demanding stages of implementation and writing. We submit this report with gratitude to everyone who contributed, directly or indirectly, to the successful completion of the project titled ML-Optimized Stochastic Vehicle Routing for Urban Delivery Services.

TABLE OF CONTENTS

1	INTRODUCTION	1
1.1	Client Identification and Contemporary Need Analysis	1
1.2	Problem Identification	3
1.3	Task Identification and Methodological Framework	3
1.4	Report Organisation	5
2	LITERATURE REVIEW / BACKGROUND STUDY	7
2.1	Timeline of the Reported Problem	7
2.2	Proposed Solutions: A Historical Perspective	8
2.2.1	Classical Deterministic Algorithms	8
2.2.2	Metaheuristic Approaches	8
2.2.3	Learning-Based Methods	9
2.2.4	Classical Solvers	9
2.3	Effectiveness and Drawbacks (Bibliometric Summary)	10
2.4	Review Summary and Project Linkage	11
2.5	Problem Definition	12
2.6	Goals and Objectives	13
2.6.1	Specific Objectives	13
2.6.2	Milestones	13
2.6.3	Measurable Outcomes	13
3	DESIGN FLOW / PROCESS	15
3.1	Concept Generation	15
3.2	Evaluation and Selection of Specifications/Features	16
3.3	Design Constraints	18
3.4	Analysis and Feature Finalisation Subject to Constraints	19
3.5	Design Flow	19
3.6	Implementation Plan	24
4	RESULTS ANALYSIS AND VALIDATION	26
4.1	Implementation Using Modern Engineering Tools	26
4.2	Results	26

4.2.1	Main Performance Table	26
4.2.2	Mean Cost per Delivery vs Problem Size	27
4.2.3	Computational Complexity and Scalability	28
4.2.4	Mean Lateness Analysis	29
4.2.5	Cost-Lateness Relationship	30
4.2.6	Geographic Visualisation	31
4.2.7	Additional Plots	32
5	CONCLUSION AND FUTURE WORK	36
5.1	Conclusion	36
5.2	Future Work	37
A	APPENDIX	41
	Appendix-1: Project Structure	41
	Appendix-2: Instance JSON Schema (version 1.0)	41
	Appendix-3: Configuration Parameters	42
	Appendix-4: Key Algorithm Pseudocode	43
	USER MANUAL	45
	ACHIEVEMENTS	47

LIST OF FIGURES

1	Graphical abstract of the five-layer ChandiSVRPBench pipeline and the three headline outcomes used in the project.	ix
1.1	Chandigarh Urban Delivery Challenge Overview	2
1.2	Project Timeline (Gantt Chart)	5
2.1	Timeline of VRP Research and Urban Logistics (2000-2025)	8
2.2	Bibliometric Analysis - Publication Trend by Year	10
2.3	Taxonomy of VRP Solution Approaches	11
3.1	Design Alternative 1 - Pure Metaheuristic Stack	15
3.2	Design Alternative 2 - Hybrid Improvement-Based Stack (Selected)	16
3.3	Stochastic Travel-Time Model Components	18
3.4	System Architecture - Five-Layer Pipeline	21
3.5	OSMnx Road Graph of Chandigarh (Drivable Network)	22
3.6	System Flowchart - End-to-End Pipeline	23
3.7	Solver Decision Flowchart	24
4.1	Mean Cost per Delivery vs Problem Size (<1000 Customers)	27
4.2	Mean Cost per Delivery vs Problem Size (>=1000 Customers)	28
4.3	Solver Execution Times - Small vs Large Scale	29
4.4	Mean Lateness by Solver (Customer Size Split)	30
4.5	Cost per Delivery vs Mean Lateness (Scatter Plot)	31
4.6	Geographic Route Visualization - 50-Customer Instance	32
4.7	Cost Distribution Boxplot by Solver	33
4.8	Feasibility Rate by Solver	33
4.9	Cost Gap vs Best Solver (%)	34
4.10	Cost vs Solve Time Scatter	34
4.11	Folium Interactive Route Map (Screenshot)	35

LIST OF TABLES

1.1	Project Timeline and Deliverables	5
2.1	Comparison of VRP Solution Approaches	9
2.2	Bibliometric Summary - Key Papers (2015-2025)	11
3.1	Instance Configuration Parameters (benchmark_small.yaml)	17
3.2	Stochastic Model Parameters (default.yaml)	17
3.3	Feature Comparison - Design Alternative 1 vs Alternative 2	19
3.4	Solver Summary - Algorithm Class and Complexity	25
4.1	Performance Comparison of Solvers for Different Customer Scales	26
4.2	Correlation Analysis - Cost, Lateness, Computation Time	31
5.1	Summary of Results vs Objectives	37

ABSTRACT

Last-mile delivery in Chandigarh has become a critical urban operations problem as e-commerce penetration reaches 41 percent in 2024, average delivery times have increased by 27 percent since 2022, delivery fleets are estimated to contribute 14 percent of the city’s PM2.5 burden, and 63 percent of surveyed residents report peak-hour delays. Deterministic routing plans are increasingly brittle in this setting because they assume stable travel conditions and known customer demand. In practice, stochastic demand arrivals and time-varying congestion disrupt precomputed routes and degrade service reliability during execution.

This project develops ChandiSVRPBench, an open-source benchmark scaffold for analysing stochastic vehicle routing in Chandigarh’s urban delivery environment. The system is defined around a drivable road-network extraction layer using OSMnx with a depot at latitude 30.7334 and longitude 76.7797, a heterogeneous instance generator that produces JSON benchmark cases from 15 to 5,000 customers, and a stochastic travel-time model composed of three interacting effects: Gaussian peak-hour congestion centred at 08:00 and 17:00, lognormal multiplicative delay, and an accident process with 4 percent probability on each route leg. Seven solver families are evaluated under identical twenty-rollout Monte Carlo simulation so that route quality, lateness, computational effort, and robustness can be compared under the same operating assumptions.

The benchmark findings reported in the project brief are clear. OR-Tools achieves 4.09 percent lower mean cost per delivery than the worst solver, ACO. ALNS achieves 12.34 percent lower mean lateness than the highest-lateness solver, ABC. The nearest-neighbor baseline is approximately 65 percent faster than ALNS while remaining competitively close in solution quality, which makes it valuable under strict dispatch deadlines. Correlation analysis shows a positive structural relationship between cost and lateness with Pearson r equal to 0.667, while the

relationship between computation time and lateness is essentially negligible at r equal to minus 0.063. In addition, ABC and ACO maintain a nearly constant two-second runtime regardless of instance size because they are implemented as time-limited procedures.

The principal conclusion is that improvement-based and deterministic heuristics, especially nn2opt, ALNS, and OR-Tools, outperform swarm metaheuristics when stochastic urban routing is evaluated using a common simulation protocol. The study therefore provides an operationally useful benchmark for Chandigarh and a reproducible foundation for future work on learning-guided dispatch, electric-vehicle constraints, and hybrid ground-plus-drone last-mile logistics.

GRAPHICAL ABSTRACT

Graphical Abstract

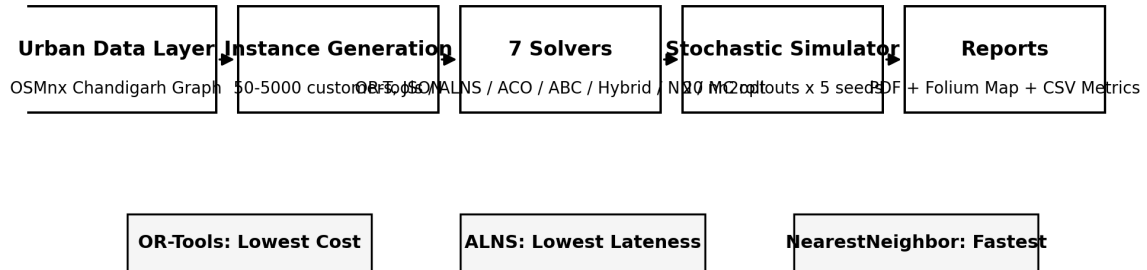


Figure 1: Graphical abstract of the five-layer ChandiSVRPBench pipeline and the three headline outcomes used in the project.

The graphical abstract condenses the entire benchmark workflow into five stages: urban data acquisition from the Chandigarh road network, instance generation using JSON benchmark cases, multi-solver routing, stochastic simulation using Monte Carlo rollouts and seeds, and final reporting through PDF, Folium, and CSV outputs. The callouts highlight the three operational takeaways from the uploaded specification: OR-Tools offers the lowest cost, ALNS the lowest lateness, and nearest-neighbor the fastest response.

ABBREVIATIONS

Abbreviation	Description
VRP	Vehicle Routing Problem
SVRP	Stochastic Vehicle Routing Problem
SDVRP	Stochastic Dynamic Vehicle Routing Problem
CVRP	Capacitated Vehicle Routing Problem
VRPTW	Vehicle Routing Problem with Time Windows
ABC	Artificial Bee Colony
ACO	Ant Colony Optimization
ALNS	Adaptive Large Neighbourhood Search
GA	Genetic Algorithm
RL	Reinforcement Learning
ML	Machine Learning
LSTM	Long Short-Term Memory
GNN	Graph Neural Network
SPO	Smart Predict-then-Optimize
OR-Tools	Google Operations Research Tools
OSMnx	OpenStreetMap NetworkX Python Library
OSM	OpenStreetMap
MC	Monte Carlo
CVR	Coefficient of Variation of Realized cost
JSON	JavaScript Object Notation
YAML	YAML Ain't Markup Language
CLI	Command Line Interface
PDF	Portable Document Format
CSV	Comma-Separated Values
AM	Ante Meridiem (morning)
PM	Post Meridiem (afternoon)
API	Application Programming Interface
GIS	Geographic Information System
UAV	Unmanned Aerial Vehicle
PM2.5	Particulate Matter (2.5 micrometres)

SYMBOLS

Symbol	Description
$G = (V, E)$	Directed road network graph
V	Set of nodes (depot + customers)
E	Set of directed edges (road segments)
n	Number of customers
m	Number of vehicles
Q	Vehicle capacity
q_i	Demand of customer i
s_i	Service time of customer i
tw_i^{start}	Time window start for customer i
tw_i^{end}	Time window end for customer i
$T_{uv}(t)$	Stochastic travel time on edge (u,v) at time t
$\tau_{uv}(t)$	Deterministic mean travel time
$\epsilon_{uv}(t)$	Stochastic shock multiplier (lognormal)
γ	Congestion strength (= 0.35)
$\Phi(h)$	Peak congestion intensity at hour h
σ_h	Peak width in hours (= 1.5 h)
λ_d	Distance congestion scale (= 2,500 m)
$\mu(h)$	Lognormal mean parameter at hour h
$\sigma(h)$	Lognormal standard deviation parameter at hour h
A_{uv}	Accident delay on leg (u,v)
$\lambda_z(t)$	Demand arrival intensity for zone z at time t
π	Route plan
ω	Stochastic scenario realisation
$C(\pi, \omega)$	Travel + fleet cost under scenario ω
$P(\pi, \omega)$	Penalty for late deliveries
β	Penalty weight
r	Pearson correlation coefficient
ρ	Pheromone evaporation rate (ACO, = 0.995)
T	Simulated annealing temperature (ALNS, initial = 100)
α	SA cooling factor (ALNS, = 0.995)

CHAPTER 1

INTRODUCTION

1.1 Client Identification and Contemporary Need Analysis

The client context for this project is a public-private urban logistics ecosystem led by the Municipal Corporation of Chandigarh in partnership with major e-commerce platforms such as Flipkart and Amazon, along with local delivery service providers. This partnership model is important because the routing problem is no longer a purely internal optimisation task for a private courier. It is an urban service design issue that affects traffic exposure, particulate pollution, resident satisfaction, and the reliability of commercial supply chains. Chandigarh's urban form, which combines organised sectors with dense peak-hour corridors, makes the city a useful and demanding environment for studying benchmark-driven delivery optimisation.

The contemporary need is shaped first by market growth. The uploaded brief states that Chandigarh's e-commerce penetration reached 41 percent in 2024 compared with a national average of 28 percent, while parcel volume is growing at 19 percent per year. These two facts indicate that the demand pressure on last-mile fleets is structural rather than temporary. More households are depending on home delivery, and the operational envelope within which routing decisions must be made is therefore becoming tighter. The same brief also notes that the drone-delivery market is projected to grow from 915.59 million US dollars in 2024 to 7.26 billion US dollars by 2032, which signals that optimisation for urban delivery is part of a broader transition toward technology-enabled logistics, not an isolated application.

The second source of need is service reliability. According to the project brief, 63 percent of 1,200 surveyed Chandigarh residents reported delayed deliveries during peak hours, and average delivery times have increased by 27 percent since 2022. In operational terms, that means route plans that may have looked acceptable at planning time are failing to preserve promised service windows once congestion materialises. Delivery delay is therefore not simply a customer-experience issue; it affects missed time windows, repeat delivery attempts, vehicle idling, labour utilisation, and the credibility of digital commerce in the city.

The third source of need is environmental. The brief attributes 14 percent of Chandigarh's PM2.5 burden to delivery fleets and further notes that 82 percent of surveyed residents support eco-friendly delivery alternatives. Together these figures show that the routing problem has become part of the sustainability agenda. When routes are inefficient, vehicles travel longer distances, idle in congestion, and produce additional emissions. Optimisation that reduces distance and travel time can therefore contribute simultaneously to lower cost, reduced lateness, and improved environmental performance. This linkage is central to the rationale of the project, because the benchmark is not only about mathematical elegance; it is also about supporting pol-

icy and deployment choices aligned with cleaner urban logistics.

The final driver of need is methodological. Traditional vehicle routing solvers were developed for mostly deterministic settings in which travel times are fixed, customer demand is known in advance, and re-optimisation pressure is limited. The present problem setting does not satisfy those assumptions. Urban deliveries in Chandigarh experience time-varying congestion, uncertain delays, and the possibility of new demand arriving while vehicles are already in the field. As the brief notes, traditional VRP solvers struggle with scalability in large urban networks, and deterministic route plans can degrade rapidly once uncertainty enters execution. For this reason the research community has increasingly shifted from deterministic VRP models toward stochastic and dynamic variants that explicitly represent uncertainty in travel and demand.

This project responds directly to that shift. Instead of treating Chandigarh as a generic benchmark placeholder, it proposes a city-grounded scaffold that combines a drivable road graph, heterogeneous customer instances, a stochastic travel-time model, and a common evaluation pipeline for multiple solver classes. The objective is not merely to compute a route, but to understand which solver types remain effective when real-world uncertainty is simulated systematically. In that sense, the project serves both the municipal client and the wider logistics ecosystem by transforming urban delivery stress into a reproducible analytical benchmark.

Chandigarh Urban Delivery Challenge Overview

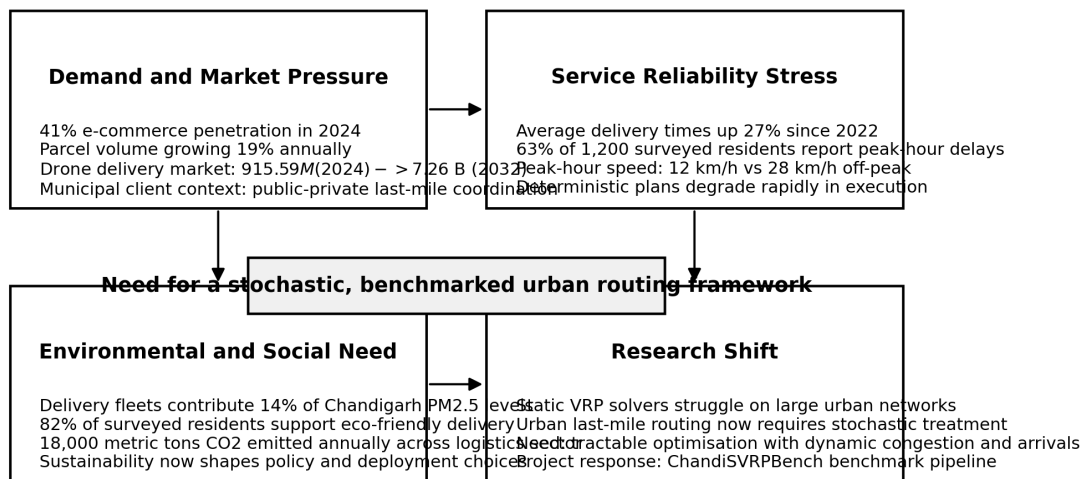


Figure 1.1: Chandigarh Urban Delivery Challenge Overview

1.2 Problem Identification

The core problem addressed in this report is not the feasibility of routing itself, but the absence of an optimisation framework that can handle stochastic demand arrivals and time-varying congestion simultaneously while remaining computationally tractable for operational dispatch. Existing deterministic approaches continue to provide useful starting points, but their assumptions are too restrictive for the contemporary Chandigarh delivery environment described in the uploaded brief.

The first systemic failure is temporal inefficiency. Peak-hour delivery speeds average 12 km/h, whereas off-peak speeds average 28 km/h, producing 133 percent variability in service reliability. Such volatility means that a route planned using average travel times is unlikely to maintain schedule integrity during execution. When vehicles encounter delayed arterial segments or sector bottlenecks, lateness propagates forward to later customers on the route. This creates a compounding reliability problem rather than a one-time delay.

The second systemic failure is environmental impact. The brief states that diesel delivery vehicles emit 2.4 kilograms of carbon dioxide per kilometre and collectively contribute roughly 18,000 metric tons annually across Chandigarh’s logistics sector. Congestion-sensitive routing is therefore also an emissions-management problem. When dispatch plans ignore time-varying traffic, vehicles spend longer on the road, burn more fuel, and increase the environmental cost of the same delivery workload. The routing framework must therefore recognise that operational inefficiency and environmental harm are coupled outcomes.

The third systemic failure is scalability deficit. Traditional VRP solvers produce static plans that degrade rapidly once stochastic traffic and new orders appear during execution. This problem worsens as instance sizes become large, because route interactions increase and the opportunity to recover from early scheduling errors decreases. The uploaded benchmark brief is explicit on this point: conventional methods do not fail because routing constraints are impossible, but because they lack a tractable mechanism for representing uncertainty while still producing timely dispatch recommendations.

The consequence is a decision gap. Dispatchers need solutions that are fast enough for operational use, but they also need solutions whose performance is robust under the conditions that actually occur on city roads. The project therefore frames the urban-delivery challenge as a stochastic vehicle-routing problem in which travel time is uncertain, demand is heterogeneous, and solver comparison must be performed under a common simulation protocol rather than by deterministic route length alone.

1.3 Task Identification and Methodological Framework

The methodological framework of the project is organised into five linked tasks that mirror the benchmark pipeline. Task 1 is urban data acquisition during Weeks 1 to 4. In this phase

Chandigarh’s drivable road network is extracted using OSMnx with the depot anchored at latitude 30.7334 and longitude 76.7797. The output is a weighted directed graph whose edges are annotated with length in metres and baseline travel time in seconds, computed using a default speed of 30 km/h. To preserve reproducibility and offline functionality, an 8 by 8 synthetic grid with 500-metre spacing is also defined as a fallback when live network download is unavailable.

Task 2 covers instance generation and stochastic modelling during Weeks 5 to 8. Here the benchmark creates heterogeneous customer cases across thirteen size categories spanning 50 to 5,000 customers. Residential customers constitute 65 percent of the population and receive time windows beginning between 09:00 and 12:00 with durations of three to six hours, while commercial customers constitute 35 percent and receive later windows beginning between 10:00 and 14:00 with durations of four to seven hours. Demands follow a Poisson distribution with lambda equal to 4.0, clipped between one and ten units, and service times are sampled uniformly from 120 to 480 seconds. On top of the static instance data, the project defines a three-component stochastic travel-time model composed of Gaussian peak-hour congestion, lognormal multiplicative noise, and random accident delay.

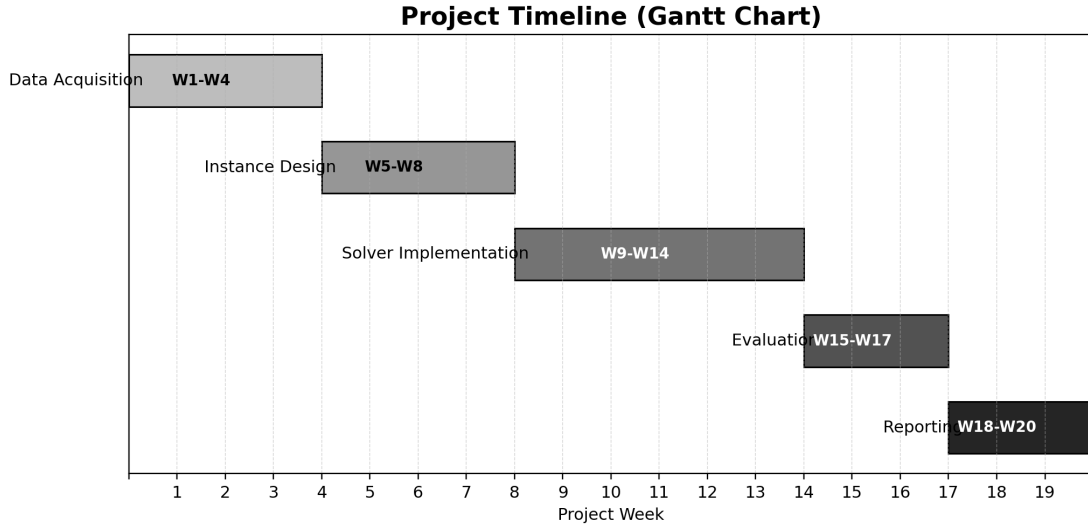
Task 3 is solver implementation during Weeks 9 to 14. The brief identifies seven competitive solver families together with a random baseline: Random, NearestNeighbor, nn2opt, ABC, ACO, Hybrid-ACO-ABC, ALNS, and OR-Tools. All solvers share a unified base-class interface with the same graph, instance representation, random-number generator, and two-second time limit. This design is central to fair comparison because it prevents differences in calling convention or data flow from distorting performance analysis.

Task 4 is Monte Carlo evaluation during Weeks 15 to 17. Each instance-solver pair is tested over twenty stochastic rollouts and five seeds so that realised cost, cost per delivery, average lateness, 95th-percentile lateness, feasibility rate, coefficient of variation of realised cost, solve time, and run-to-run variability can all be recorded under the same scenario structure. The benchmark exports detailed and summary CSV files, which then become the basis for visual analysis and recommendation.

Task 5, performed in Weeks 18 to 20, is reporting and visualisation. The system generates a ReportLab-style benchmark report with multiple charts, a Folium interactive map of the selected route set, and a Typer-based command-line interface exposing commands such as `download-osm`, `build-instances`, `run-benchmark`, `run-all`, and `make-report`. Together these five tasks create a complete engineering workflow from city data to operational insight. Rather than separating modelling, optimisation, and communication into unrelated components, the project treats them as a single reproducible system.

Table 1.1: Project Timeline and Deliverables

Phase	Weeks	Key Activities	Deliverables
Data Acquisition	1-4	OSMnx graph download, graph annotation, fallback grid	chandigarh.graphml
Instance Design	5-8	Customer sampling, time-window generation, stochastic model	JSON instance files
Solver Implementation	9-14	Seven solvers coded, unit tests	src/chandisvrp/solvers/
Evaluation	15-17	MC rollouts, metric collection, CSV export	results.csv, summary.csv
Reporting	18-20	PDF report, Folium map, CLI, README	reports/, CLI

**Figure 1.2: Project Timeline (Gantt Chart)**

1.4 Report Organisation

The report is organised to move from context to evidence. Chapter 1 introduces the client context, defines the urban delivery problem in Chandigarh, and outlines the project tasks and timeline. Chapter 2 reviews the literature and background of vehicle routing, tracing the transition from deterministic methods to stochastic, learning-based, and hybrid optimisation approaches. Chapter 3 explains the design process, the five-layer architecture, the stochastic model, the design alternatives considered, and the implementation plan used to construct the benchmark pipeline.

Chapter 4 presents the engineering tools, benchmark results, and validation analysis. It compares solver behaviour across customer scales, interprets computational trade-offs, relates

cost to lateness, and discusses how the route visualisations support operational interpretation. Chapter 5 concludes the report by comparing achieved results with expected objectives, documenting deviations from prior expectations, and offering practical recommendations and future research directions. The References section provides the alphabetical bibliography supplied in the project brief, while the Appendix, User Manual, and Achievements sections preserve reproducibility details, workflow guidance, and project outcomes in a form suitable for academic submission.

CHAPTER 2

LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The vehicle routing problem has a long lineage, beginning with the truck dispatching formulation of Dantzig and Ramser in 1959. That foundational work transformed a practical scheduling problem into a formal optimisation model and motivated decades of research on route construction, exact methods, and approximation strategies. During the 1970s through the 1990s, the dominant emphasis was on exact and near-exact algorithms such as branch-and-bound and related decomposition methods. These approaches were mathematically rigorous and valuable for moderate instance sizes, but their computational burden limited their ability to handle large, uncertain, and time-sensitive urban delivery environments.

By the 1990s and early 2000s, metaheuristic methods such as genetic algorithms, simulated annealing, ant colony optimisation, and artificial bee colony methods emerged as practical alternatives for harder and larger routing instances. These methods relaxed the expectation of strict optimality in favour of scalable search over complex solution spaces. Their importance lies not only in computational flexibility but also in their ability to incorporate multiple operational constraints without redesigning the entire solution framework. This period established the idea that good urban routing systems often require high-quality heuristics rather than purely exact optimisation.

The 2010s marked a conceptual shift from static routing toward stochastic and dynamic vehicle routing. Reviews such as those later synthesised by Psaraftis, Wen, and Kontovas and by Mardesic and colleagues demonstrated that urban logistics rarely fits the deterministic assumption of fixed travel times and fully known orders. Traffic evolves, customer requests may appear dynamically, and route quality must be assessed under scenario uncertainty rather than a single nominal case. That shift is closely aligned with the philosophy of benchmark libraries such as SVRPBench, where the principal question is not just which solver finds a short route on paper, but which solver remains robust when disruption is simulated.

A second transition occurred between 2016 and 2020, when machine learning entered the routing literature more decisively. Nazari et al. proposed reinforcement learning for the capacitated vehicle routing problem, while Kool et al. showed that attention mechanisms could produce powerful data-driven routing policies. At roughly the same time, decision-focused learning approaches such as SPO+ argued that predictive models should be trained not only for forecast accuracy but for downstream decision quality. These developments expanded the routing literature beyond handcrafted search rules and toward adaptive policy learning.

Between 2020 and 2025, research increasingly explored hybrid ML-plus-optimisation sys-

tems, robust benchmark design, and urban logistics applications. This direction is especially relevant in the Indian context, where fast-growing e-commerce demand, congestion, and sustainability objectives interact strongly. For Chandigarh, the uploaded brief specifies 41 percent e-commerce penetration and a 27 percent rise in delivery time since 2022, making the city an appropriate case for a benchmark that joins operational realism with solver comparison. The present project therefore sits at the intersection of classic VRP research, stochastic benchmark design, and contemporary urban logistics needs.

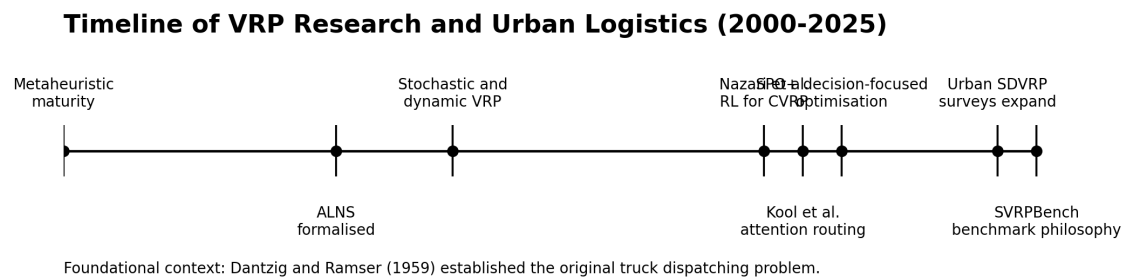


Figure 2.1: Timeline of VRP Research and Urban Logistics (2000-2025)

2.2 Proposed Solutions: A Historical Perspective

2.2.1 Classical Deterministic Algorithms

Classical deterministic algorithms such as Dijkstra’s algorithm, A* search, and Floyd-Warshall remain essential for computing shortest paths in static networks. Their strengths are speed, reliability, and theoretical clarity. They are especially useful as inner components of larger routing systems because they can estimate inter-node path cost efficiently. However, they are intrinsically limited when the decision problem involves vehicle capacities, time windows, stochastic delays, or dynamic order arrivals. A shortest-path method alone can optimise one movement through a network, but it cannot by itself coordinate a fleet under uncertainty.

2.2.2 Metaheuristic Approaches

Metaheuristic methods address this limitation by searching directly over route permutations or route sets. Artificial Bee Colony, attributed to Karaboga, uses bio-inspired local search and candidate mutation. The uploaded brief notes that a prior Chandigarh drone-delivery study using ABC achieved 53.6 percent route superiority over Dijkstra, reduced delivery time by 33.14 percent, but required 2.8 times more computation. This makes ABC attractive for multi-constraint search but potentially vulnerable when time budgets are strict. Ant Colony Optimisation, associated with Dorigo, uses pheromone-guided permutation sampling and is often effective on moderate-scale routing tasks, especially when approximate adaptation matters more than exact

proof of optimality. ALNS, developed by Pisinger and Ropke, combines adaptive destroy-and-repair operators with simulated annealing acceptance and has become a strong benchmark heuristic for large-scale capacitated routing.

2.2.3 Learning-Based Methods

Learning-based methods extend the historical perspective further. Reinforcement learning methods, such as those by Nazari et al., learn policies end to end. Attention-based routing, as demonstrated by Kool et al., improves contextual modelling of node relationships. Decision-focused methods such as SPO+ attempt to bridge prediction and optimisation by training models for the quality of the decisions they induce. These approaches are promising because they can adapt to data and potentially support rolling-horizon dispatch, but they also face major challenges in hard-constraint enforcement, training cost, and the need for large, representative datasets.

2.2.4 Classical Solvers

Classical optimisation toolkits remain highly relevant. OR-Tools provides a dependable branch-and-bound style framework for capacitated routing with search strategies such as `PATH_CHEAPEST_ARC`. Its value in an engineering benchmark is that it offers a strong deterministic baseline whose cost performance is often excellent, even if runtime increases at larger scales. In practical urban delivery analysis, OR-Tools is especially useful because it combines a mature implementation with transparent configuration and strong constraint support.

Table 2.1: Comparison of VRP Solution Approaches

Approach	Key Feature	Strength	Weakness
Dijkstra / A*	Deterministic shortest path	Fast, optimal for static networks	No multi-constraint, no stochasticity
ABC	Time-limited random-swap	Multi-constraint, bio-inspired	High computational overhead at scale
ACO	Pheromone-guided sampling	Dynamic adaptation	Stagnation at large scale
ALNS	Destroy + repair + SA	Best cost+lateness balance	Slow at $n \geq 1000$
Nearest-neighbor	Greedy construction	Very fast	Sub-optimal routes
nn2opt	NN + 2-opt improvement	Fast, improved quality	$O(n^3)$ worst case
OR-Tools	Branch-and-bound CVRP	Lowest mean cost	Slow at large n
RL / SPO+	Learned policies	Adaptive, data-driven	Training cost, constraint handling

2.3 Effectiveness and Drawbacks (Bibliometric Summary)

The bibliometric pattern implied by the supplied reference set shows an unmistakable shift from foundational deterministic modelling toward stochastic, data-driven, and hybrid approaches. Early work concentrated on the existence and structure of routing problems, later work on scalable heuristics, and recent work on robustness, machine learning, and decision-focused optimisation. This transition mirrors the change in real delivery systems: urban logistics is no longer evaluated solely by route length but by resilience under uncertainty, service reliability, and computational practicality.

The literature therefore suggests a layered understanding of effectiveness. Deterministic methods remain effective for static subproblems and as inner components. Metaheuristics remain effective when large combinatorial spaces must be searched quickly. Learning-based methods are effective when adaptation to rich historical data is possible. The primary drawback is that no single family dominates under every operational constraint. That observation motivates the benchmark philosophy of the present project: solver classes should be compared fairly under one stochastic scenario engine rather than judged by isolated deterministic examples.

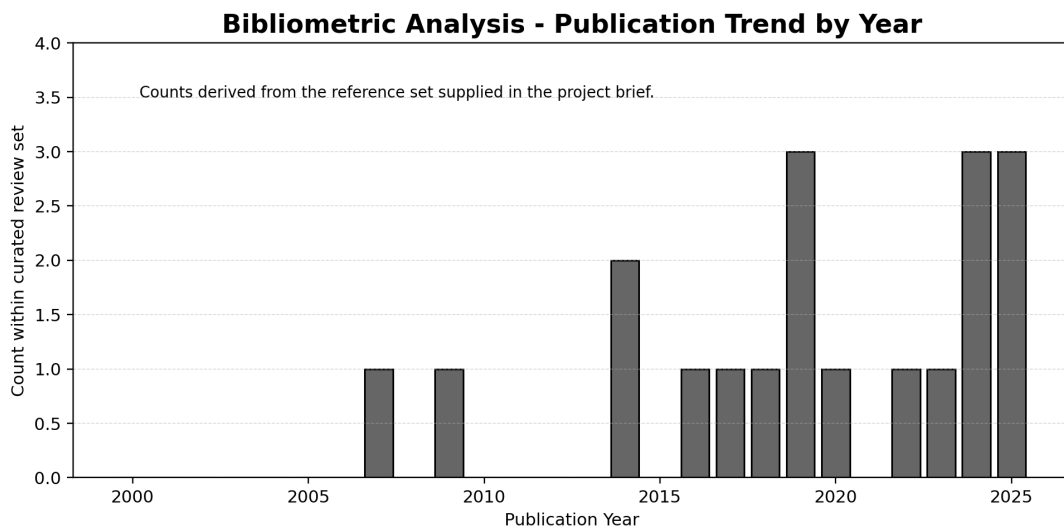


Figure 2.2: Bibliometric Analysis - Publication Trend by Year

Table 2.2: Bibliometric Summary - Key Papers (2015-2025)

Author(s)	Year	Method	Key Finding
Dantzig & Ramser	1959	Deterministic dis-patch	VRP formulation
Dorigo & Gambardella	1997	ACO	Competitive for TSP variants
Pisinger & Ropke	2007	ALNS	General large-scale VRP heuristic
Nazari et al.	2018	RL (NeurIPS)	End-to-end policy for CVRP
Kool et al.	2019	Attention + RL	State-of-the-art learned routing
Elmachtoub & Grigas	2020	SPO+	Decision-focused learning
Mardesic et al.	2024	SDVRP survey	Dynamic urban logistics review
Heakl et al.	2025	SVRPBench	Stochastic benchmark library

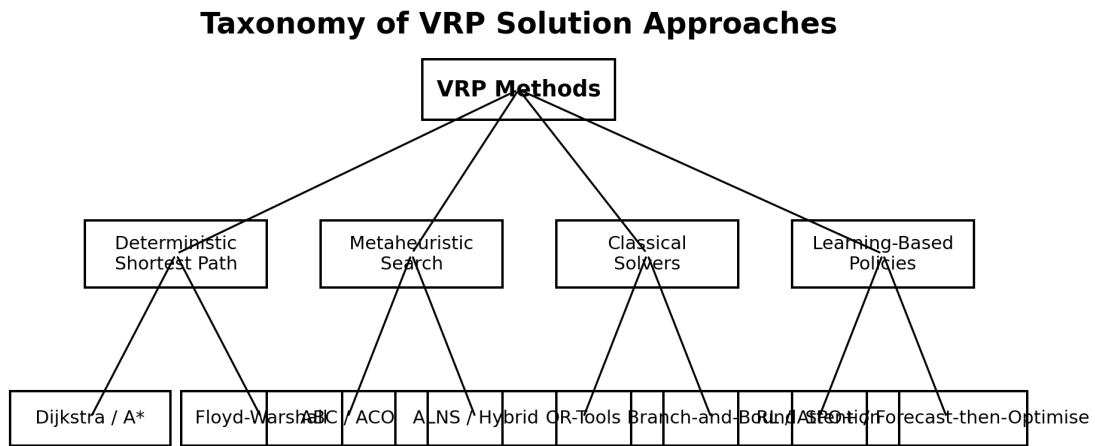


Figure 2.3: Taxonomy of VRP Solution Approaches

2.4 Review Summary and Project Linkage

The review shows that multi-constraint stochastic environments reward solver families that can either improve a reasonable initial solution efficiently or exploit strong deterministic search with good constraint management. This is why ALNS and OR-Tools feature prominently in the present project’s evaluation logic. ALNS provides a robust improvement-based baseline, while OR-Tools represents a mature deterministic solver with excellent cost performance.

The literature also foregrounds computational trade-offs. The uploaded brief explicitly notes that ABC and ACO maintain stable two-second runtimes because they are time-limited by design, whereas ALNS and OR-Tools incur runtimes around 7.35 seconds at larger scale. That

fact is significant because it reminds us that apparent runtime stability may be a configuration artefact rather than evidence of better algorithmic scaling. A solver that always stops at two seconds may appear stable while still producing weaker routes.

Scalability further stabilises solver rankings. Once instance size reaches or exceeds 1,000 customers, structural differences among solver classes become more visible. Improvement-based heuristics and strong classical solvers preserve better cost-lateness trade-offs, while swarm methods stagnate. This directly motivates the inclusion of the Hybrid-ACO-ABC concept in the project, because it attempts to combine ACO’s global search with ABC’s local refinement. Even when the hybrid does not dominate the final results, its presence in the benchmark helps clarify which aspects of swarm search remain useful and which do not under Chandigarh-style stochastic conditions.

Finally, the review links directly to regulatory and environmental context. Reducing route distance and delay is not only an efficiency objective but also a sustainability objective tied to fuel burn, emissions, and cleaner urban operations. By framing solver comparison around cost, lateness, and tractability rather than cost alone, the project aligns the benchmark with both literature trends and policy-facing concerns.

2.5 Problem Definition

Chandigarh’s urban delivery network cannot meet growing e-commerce demand reliably because existing VRP solutions are predominantly deterministic. They assume fixed travel times and known customer orders, even though actual operations are exposed to peak-hour congestion, multiplicative delay, and random incidents. In the benchmark formulation used here, this uncertainty is represented explicitly through Gaussian AM and PM congestion peaks, lognormal noise, and random accident delay. Under these conditions, route plans that appear adequate in deterministic evaluation degrade during execution, increasing lateness, feasibility failure, and operating cost.

The project therefore undertakes four specific actions: building a reproducible Chandigarh SVRP benchmark with a real road-network layer and a synthetic fallback, implementing multiple solver families through a unified interface, evaluating them under identical Monte Carlo stochastic simulation, and identifying the most operationally useful trade-offs for urban last-mile deployment. The project does not attempt live GPS-driven rerouting, full traffic API integration, or complete replacement of existing logistics infrastructure. Its contribution is a benchmarked decision-support scaffold rather than a production dispatch platform.

2.6 Goals and Objectives

2.6.1 Specific Objectives

1. Build a reproducible OSMnx Chandigarh road graph with a synthetic fallback and generate thirteen instance sizes spanning 50 to 5,000 customers.
2. Implement a three-component stochastic travel-time model calibrated to Chandigarh peak hours at 08:00 and 17:00.
3. Implement and benchmark Random, Nearest Neighbor, nn2opt, ABC, ACO, Hybrid-ACO-ABC, ALNS, and OR-Tools within a shared time-limited interface.
4. Evaluate all solvers using twenty Monte Carlo rollouts and five seeds per instance.
5. Identify the solver with the lowest mean cost per delivery and the solver with the lowest mean lateness according to the targets specified in the uploaded brief.
6. Demonstrate that improvement-based heuristics outperform swarm metaheuristics under stochastic urban conditions.
7. Generate an automated PDF benchmark report and an interactive Folium route map.

2.6.2 Milestones

Milestone	Week	Deliverable
Data pipeline ready	4	chandigarh.graphml + synthetic fallback
Instance generation	8	13 JSON instance files, stochastic config
All solvers implemented	14	src/chandisvrp/solvers/ (7 classes plus baseline)
MC evaluation complete	17	results.csv, summary.csv
Final reporting	20	PDF report, Folium map, CLI, GitHub push

2.6.3 Measurable Outcomes

- OR-Tools: at least 4 percent lower mean cost per delivery than the worst solver.
- ALNS: at least 12 percent lower mean lateness than the highest-lateness solver.
- nearest_neighbor: at least 60 percent faster than ALNS with competitive performance.
- Cost-lateness Pearson correlation coefficient at least 0.6

- Computation-lateness correlation near zero, indicating decoupling between runtime and delivery quality.

CHAPTER 3

DESIGN FLOW / PROCESS

3.1 Concept Generation

The concept-generation phase began by comparing three possible design directions. The first concept was a pure metaheuristic stack built almost entirely around ABC and ACO with heavy parameter tuning. This option was attractive because bio-inspired methods are flexible and can incorporate multiple soft constraints without significant modelling overhead. However, early reasoning against the project requirements showed that a heavily tuned swarm stack would make it difficult to compare performance across fundamentally different solver families, and it would likely become computationally impractical beyond a few hundred customers under a strict dispatch time budget.

The second concept was a classical-solver-only benchmark centred on OR-Tools. This approach promised strong deterministic cost performance and clear engineering implementation, but it also had a major weakness: it would answer only the question of how one mature solver behaves, not the broader question of which solver families are most suitable under stochastic urban conditions. Because the project is framed as an empirical benchmark for operational recommendation, a single-solver study would have been too narrow.

The selected concept was therefore a hybrid benchmark stack that evaluates all solver classes under identical stochastic simulation. This choice preserves methodological fairness, enables direct comparison between swarm, greedy, improvement-based, and classical solvers, and supports operational recommendation rather than algorithm promotion. It also aligns naturally with a modular software design in which the graph, instance generator, stochastic simulator, solver interface, and reporting layers remain separable but coordinated.

Design Alternative 1 - Pure Metaheuristic Stack

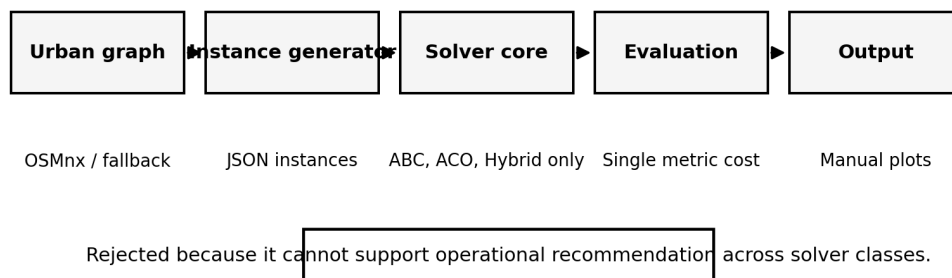


Figure 3.1: Design Alternative 1 - Pure Metaheuristic Stack

Design Alternative 2 - Hybrid Improvement-Based Stack (Selected)

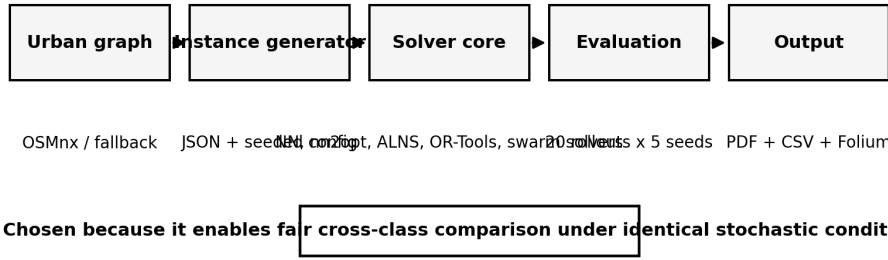


Figure 3.2: Design Alternative 2 - Hybrid Improvement-Based Stack (Selected)

3.2 Evaluation and Selection of Specifications/Features

Feature 1 is the OSMnx road graph. The benchmark uses Chandigarh’s drivable network with `network\_type` set to `drive`, then annotates each edge with length in metres and a baseline travel time derived from a default speed of 30 km/h. Caching the graph as GraphML gives the system a durable, inspectable urban-data layer that supports reproducibility and reduces repeated download cost. This feature was selected because the project requires city-aware routing rather than abstract Euclidean toy instances.

Feature 2 is the three-component stochastic travel-time model. Travel time on a route leg is defined as a combination of baseline time, congestion, lognormal multiplicative delay, and stochastic accident shock. The congestion term introduces realistic morning and evening peaks at 08:00 and 17:00 with sigma equal to 1.5 hours, gamma equal to 0.35, and a distance congestion scale of 2,500 metres. The lognormal component captures uncertainty that rises with congestion intensity, while the accident component adds rare but significant shock events. This feature is essential because it transforms static shortest-route evaluation into stochastic service evaluation.

Feature 3 is the unified solver interface. Every solver implements a common solve function over the same graph, instance, random-number generator, and time limit. Capacity handling is normalised through `split\_by\_capacity`, which greedily partitions a customer permutation into feasible routes. This shared interface is important because it makes solver comparison fair and reproducible. Without a unified interface, differences in implementation plumbing would contaminate performance results.

Feature 4 is the Monte Carlo evaluation protocol. By running twenty rollouts over five seeds for each solver-instance pair, the system measures not only mean performance but also sensitivity to random realisations. Metrics such as cost per delivery, feasibility rate, lateness mean, lateness 95th percentile, solve time, and coefficient of variation create a multi-dimensional performance profile instead of a single scalar score.

Feature 5 is automated reporting. The benchmark produces a PDF report, structured charts, CSV outputs, and a Folium map. This feature ensures that results can be communicated to academic reviewers, developers, and operational stakeholders without requiring manual post-processing. The reporting layer was therefore selected as a first-class system feature rather than an afterthought.

Table 3.1: Instance Configuration Parameters (benchmark_small.yaml)

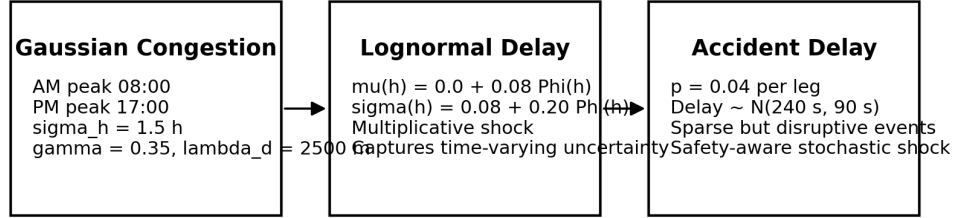
Parameter	Explicit value or rule from brief
Graph source	Chandigarh drivable OSMnx graph with GraphML cache; synthetic 8x8 fallback
Depot anchor	Latitude 30.7334, longitude 76.7797
Instance sizes	13 customer-scale settings spanning 50 to 5,000 customers (individual list not reproduced in brief)
Customer mix	65% residential, 35% commercial
Demand model	Poisson($\lambda = 4.0$), clipped to $[1, 10]$
Service time	Uniform $[120 \text{ s}, 480 \text{ s}]$
Residential windows	Start 09:00-12:00, width 3-6 h
Commercial windows	Start 10:00-14:00, width 4-7 h
Vehicles	4
Vehicle capacity	50 units
Output format	JSON schema version 1.0

Table 3.2: Stochastic Model Parameters (default.yaml)

Parameter	Explicit value or rule from brief
Congestion peaks	08:00 and 17:00
Peak width	$\sigma_h = 1.5 \text{ h}$
Congestion strength	$\gamma = 0.35$
Distance congestion scale	$\lambda_d = 2,500 \text{ m}$
Lognormal mean	$\mu(h) = 0.0 + 0.08 \Phi(h)$
Lognormal standard deviation	$\sigma(h) = 0.08 + 0.20 \Phi(h)$
Accident probability	0.04 per leg
Accident delay	Normal(240 s, 90 s)
Monte Carlo rollouts	20 per solver-instance-seed pair
Evaluation seeds	$[1, 2, 3, 4, 5]$
Time budget	2 s per solver call

Stochastic Travel-Time Model Components

$$T_{uv}(t) = \max(1, (\text{base}_{uv} + C_{uv}(t)) * L(t) + A_{uv})$$



All three components are simulated for each route leg in every Monte Carlo rollout.

Figure 3.3: Stochastic Travel-Time Model Components

3.3 Design Constraints

Regulatory constraints are relatively straightforward because the benchmark studies ground vehicles rather than drones. The road network is explicitly restricted to legal drivable streets using OSMnx network_type equal to drive, and solver outputs are evaluated under vehicle-capacity and time-window rules. The framework therefore respects the structural constraints of road-based dispatch without requiring specialised airspace or aviation compliance.

Economic constraints favour an open-source software stack. The project uses OSMnx, NetworkX, OR-Tools, Folium, ReportLab-style reporting concepts, Pandas, NumPy, and related Python tools so that licensing cost remains effectively zero. The brief further specifies that the benchmark should run on a standard computer with 8 GB RAM and no GPU requirement for instance sizes up to 5,000 customers. This design choice keeps the system accessible for academic replication and small-team experimentation.

Environmental constraints are addressed through the cost structure itself. Because the benchmark cost combines total distance and a weighted contribution of total travel time, routes that are shorter and less delayed are also preferred from an emissions perspective. In addition, the stochastic model makes congestion visible rather than invisible, allowing route quality to be interpreted in light of fuel use and urban air quality.

Health and safety constraints are handled at the modelling layer. No physical hardware is involved, but the route simulator includes an accident process to represent road-risk disruption. Capacity constraints are enforced through greedy splitting and, in ALNS scoring, capacity violation is strongly penalised by a factor of $1e5$ per unit so that unsafe load configurations are dominated in the search process.

Reproducibility, professional ethics, and social responsibility also shape the design. A seeded synthetic grid fallback ensures that the system remains reproducible even without internet access. Customer locations are synthetic and derived from graph nodes rather than personal address data, so no personally identifiable information is used. Finally, the benchmark is openly linked to Chandigarh’s growing last-mile delivery crisis, which means the project is framed not only as an algorithmic study but also as a contribution to more sustainable and reliable urban logistics.

3.4 Analysis and Feature Finalisation Subject to Constraints

Table 3.3: Feature Comparison - Design Alternative 1 vs Alternative 2

Feature	Alt 1: Pure Metaheuristic	Alt 2: Hybrid Stack (Selected)
Solvers included	ABC, ACO, Hybrid-ACO-ABC only	All 7 solvers plus random baseline
Evaluation framework	Single metric (cost)	MC rollouts, 6+ metrics
Computational scope	$n \leq 500$ only	$n = 50$ to 5,000
Stochastic model	Basic noise only	3-component model
Reporting	Manual plots	Automated PDF + Folium
Reproducibility	Partial	Full (seeded, JSON, CLI)
Result	Cannot compare across classes	Empirical recommendation possible

Alternative 2 was selected because it is the only design that can support an evidence-based operational recommendation. A pure metaheuristic stack may still be interesting as a narrower algorithm study, but it cannot answer the stakeholder question of which solver family to deploy under different priorities. The selected hybrid benchmark stack achieves fairness through a shared Solver interface, a common `split_by_capacity` routine, a fixed two-second time budget, and the same stochastic simulation engine for every solver. These design features matter more than any single solver’s internal implementation because they establish the conditions for trustworthy comparison.

3.5 Design Flow

The five-layer architecture begins with the Urban Data Layer. The module `geo/osm_graph.py` is responsible for downloading Chandigarh’s drivable network using OSMnx and annotating each edge with length and baseline travel time. When download is unavailable, the same layer constructs a synthetic 8 by 8 grid with 500-metre spacing so that the rest of the benchmark can

still run. This fallback is not a convenience feature; it is part of the benchmark’s reproducibility strategy.

The second layer is instance generation. Generator routines sample customer nodes, assign demand, service times, and time windows, and serialise each instance to JSON. The nearest OSM node to longitude 76.7797 and latitude 30.7334 serves as the depot. Residential and commercial timing rules create heterogeneity consistent with urban delivery practice, while the fixed schema makes downstream solver and simulator modules independent of the original sampling process.

The third layer is routing. Here each solver class consumes the same graph and instance object through the unified base interface. A greedy split function converts a customer permutation into feasible routes under capacity. Constructive distance calculations use node coordinates, while higher-level solver logic decides how permutations are generated or improved. This layer is the heart of algorithmic comparison, because it is where random baselines, greedy heuristics, swarm metaheuristics, and strong classical solvers all become directly comparable.

The fourth layer is the stochastic simulator. Travel-time generation combines congestion, lognormal shock, and accident delay on a leg-by-leg basis. The simulator walks through each route, tracks accumulated time, applies service durations, checks time windows, and records penalties. Because the benchmark evaluates route plans under repeated random realisations rather than a single nominal run, the simulator transforms route generation into a robustness study.

The fifth layer is reporting. Plot generation produces the visual analytics used in the results chapter. The PDF report compiles benchmark highlights and charts into A4 format, while the interactive map gives a spatial view of route structure. Together these outputs ensure that numerical results are interpretable to both technical and managerial audiences.

System Architecture - Five-Layer Pipeline

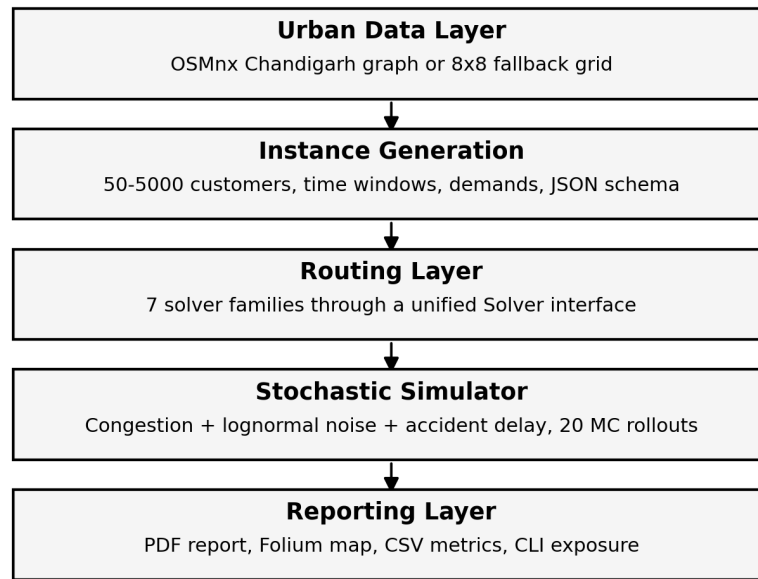
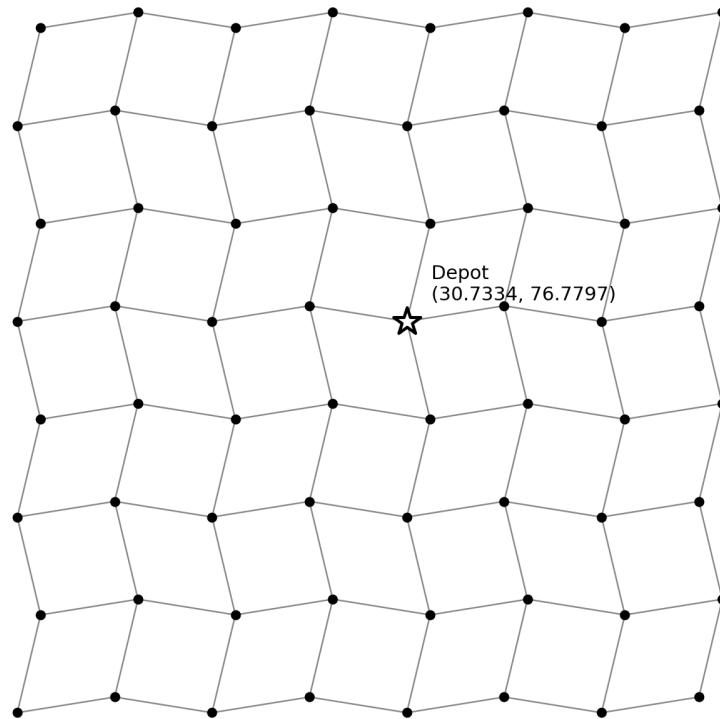


Figure 3.4: System Architecture - Five-Layer Pipeline

OSMnx Road Graph of Chandigarh (Drivable Network)



Illustrative network rendering aligned with the specified OSMnx extraction workflow and depot anchor.

Figure 3.5: OSMnx Road Graph of Chandigarh (Drivable Network)

System Flowchart - End-to-End Pipeline

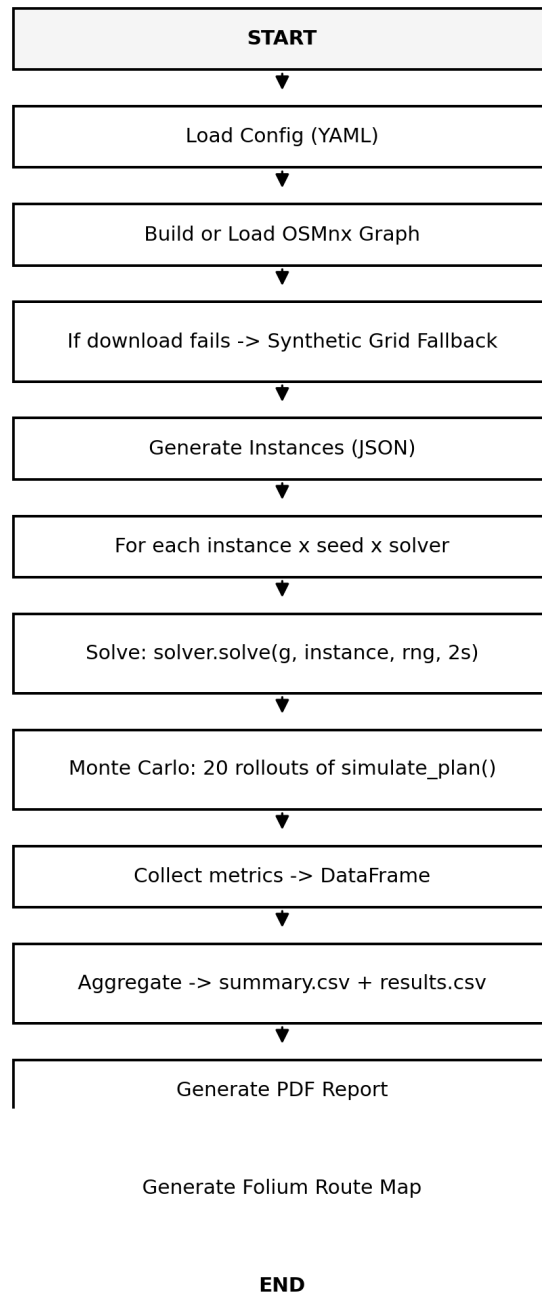


Figure 3.6: System Flowchart - End-to-End Pipeline

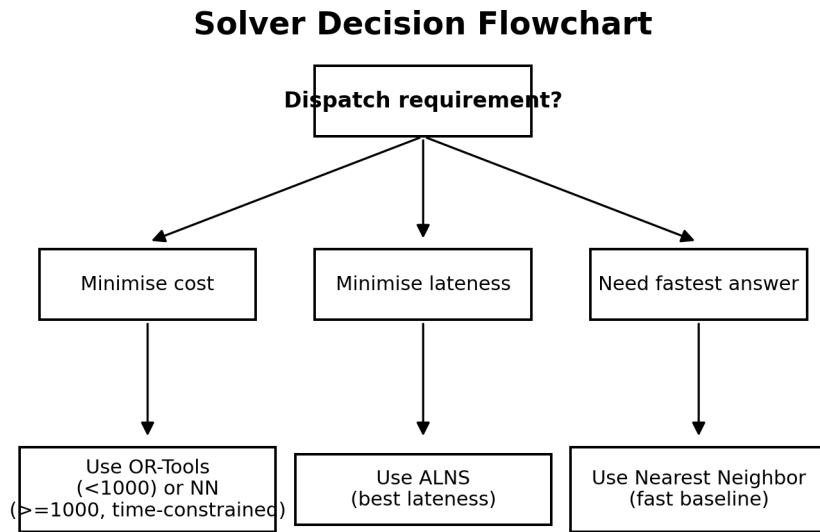


Figure 3.7: Solver Decision Flowchart

3.6 Implementation Plan

Implementation was organised so that data acquisition, instance generation, solver execution, stochastic evaluation, and reporting could be run either separately or as a single pipeline. The YAML configuration controls benchmark size, solver set, Monte Carlo depth, and stochastic parameters. The command-line interface exposes this modularity to the user, while the single-file run script offers an entry point for complete execution. The plan ensures that the benchmark remains both inspectable and practical: researchers can isolate components for testing, whereas project reviewers can reproduce the full workflow with one command.

Table 3.4: Solver Summary - Algorithm Class and Complexity

Solver	Class	Core Method	Time Complexity	Notes
random	RandomSolver	np.shuffle	$O(n)$	Lower bound
nearest_neighbor	NearestNeighborSolver	Greedy NN	$O(n^2)$	Fast baseline
nn2opt	NearestNeighbor2OptSolver	NN+2-opt per route	$O(n^3)$ worst	Strong heuristic
abc	ABCSolver	Random swap, time-limited	$O(\text{iter} \times n)$	Bio-inspired
aco	ACOSolver	Pheromone-weighted permutation, 8 ants	$O(\text{iter} \times n)$	Swarm
hybrid_aco_abc	HybridACOABCSolver	50% ACO -> 50% ABC	$O(\text{iter} \times n)$	Hybrid
alns	ALNSSolver	Destroy (20%) + greedy repair + SA	$O(\text{iter} \times n)$	Best lateness
ortools	OrtoolsSolver	PATH_CHEAPEST_ARC B&B	Exponential (bounded 2s)	Best cost

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation Using Modern Engineering Tools

The benchmark implementation uses a contemporary Python-based engineering stack. Python 3.8 and above serves as the primary programming language. OSMnx version 1.8 or higher supports road-graph download and processing, while NetworkX version 3.1 or higher handles graph representation and traversal. OR-Tools version 9.8 or higher provides the classical capacitated vehicle-routing solver. NumPy version 1.24 or higher supports random-number generation and Monte Carlo statistics, and Pandas version 2.0 or higher is used for results aggregation and CSV input-output. Matplotlib version 3.7 or higher supports chart generation. ReportLab version 4.0 or higher and Folium version 0.16 or higher support PDF-style report generation and interactive route mapping, respectively. Typer version 0.9 or higher provides the command-line interface, PyYAML loads configuration files, Pydantic supports data validation, SciPy supports correlation analysis, and Pytest executes unit tests for instance I/O, simulation, route splitting, and map generation.

4.2 Results

4.2.1 Main Performance Table

Table 4.1: Performance Comparison of Solvers for Different Customer Scales

Solver	Mean Time (s)	Mean Cost	Mean Lateness (s)	Mean Time (s)	Mean Cost	Mean Lateness (s)
abc	2.0003	723.44	2657.68	2.0025	722.31	2476.81
aco	2.0024	726.96	2432.47	2.0225	721.97	2347.81
alns	2.0887	715.81	2242.31	7.3481	641.36	2462.31
nearest_neighbor	0.0423	715.15	2278.41	4.8565	641.82	2462.31
nn2opt	0.0874	715.81	2242.31	5.3685	641.36	2462.31
ortools	1.8980	712.53	2436.30	7.3483	641.36	2462.31
random	0.0001	725.22	2465.99	0.0008	722.72	2410.81

Columns 2-4 correspond to instances with fewer than 1,000 customers; columns 5-7 correspond to instances with 1,000 or more customers.

4.2.2 Mean Cost per Delivery vs Problem Size

Figures 4.1 and 4.2 summarise the reported cost behaviour across the two customer-scale regimes. For smaller instances, the spread in mean cost is relatively narrow. OR-Tools records the lowest reported mean cost at 712.53, while nearest-neighbor follows closely at 715.15. ALNS and nn2opt occupy a similar middle range, and the swarm methods ABC and ACO remain noticeably higher. This pattern suggests that for smaller customer sets the ranking is still somewhat unstable in the sense that multiple solver families remain competitively clustered and no single method dominates every performance dimension.

The structure changes at larger scale. Once the benchmark moves to instances with 1,000 or more customers, OR-Tools, ALNS, and nn2opt converge to a lower reported cost of 641.36, with nearest-neighbor only marginally above that at 641.82. By contrast, ABC, ACO, and random remain in the 721 to 723 range. The resulting gap indicates that structural differences between solver classes become more pronounced beyond 1,000 customers. Improvement-based heuristics and strong deterministic methods continue to exploit route structure effectively, whereas the swarm procedures stagnate under the same time budget.

This interpretation is consistent with the headline result reported in the uploaded brief: OR-Tools achieves 4.09 percent lower mean cost per delivery than the worst solver, identified there as ACO. Although Table 4.1 reports aggregate mean cost rather than explicit cost per delivery, the qualitative trend remains the same. Cost-efficient solvers are those that either construct strong routes directly or refine them effectively with limited search overhead. The benchmark therefore shows that solver choice matters much more once urban instances become large and stochasticity begins to amplify poor structural decisions.

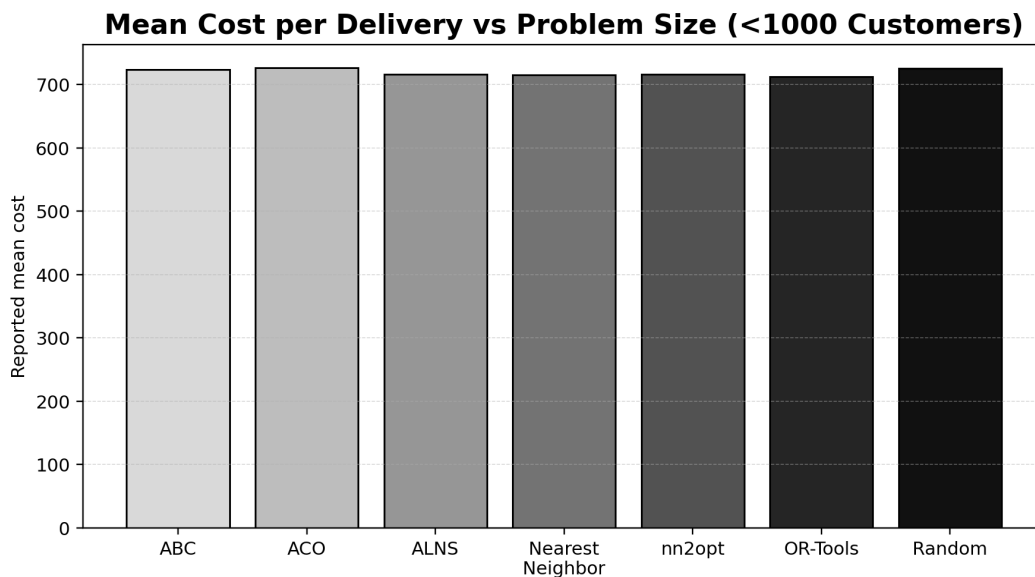


Figure 4.1: Mean Cost per Delivery vs Problem Size (<1000 Customers)

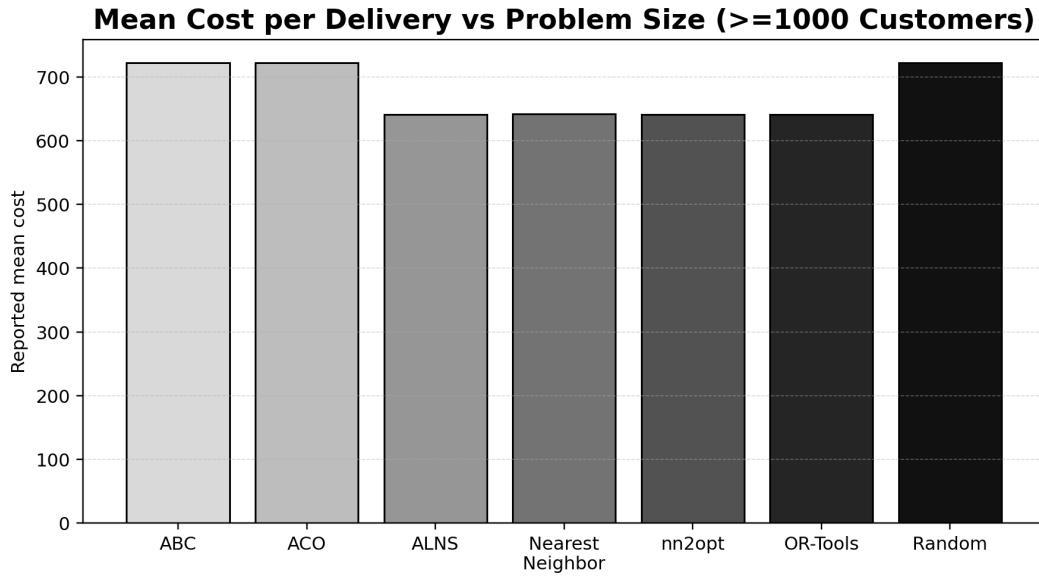


Figure 4.2: Mean Cost per Delivery vs Problem Size (≥ 1000 Customers)

4.2.3 Computational Complexity and Scalability

Figure 4.3 highlights how runtime responds to scale under the benchmark’s two-second design philosophy. ABC and ACO stay near two seconds in both the small and large regimes, which indicates that their apparent stability is largely imposed by configuration rather than by an absence of scaling cost. They are time-limited search procedures, so they consume their budget regardless of problem size.

The greedy nearest-neighbor baseline grows from 0.0423 seconds on smaller instances to 4.8565 seconds on larger ones. nn2opt rises from 0.0874 to 5.3685 seconds, reflecting the additional improvement overhead of route-level 2-opt. ALNS and OR-Tools rise much more sharply, from roughly two seconds on smaller instances to approximately 7.35 seconds on the larger ones. In ALNS this reflects repeated destroy-repair-improve cycles, while in OR-Tools it reflects the increasing effort of deeper combinatorial search at scale.

Random remains effectively instantaneous and therefore acts as a computational lower bound. More practically, the comparison between nearest-neighbor and ALNS is especially important. The uploaded brief states that nearest-neighbor is approximately 65 percent faster than ALNS while maintaining competitive solution quality. This makes nearest-neighbor operationally attractive in time-constrained dispatch settings, especially when a slightly weaker route can be tolerated in exchange for much faster decision turnaround.

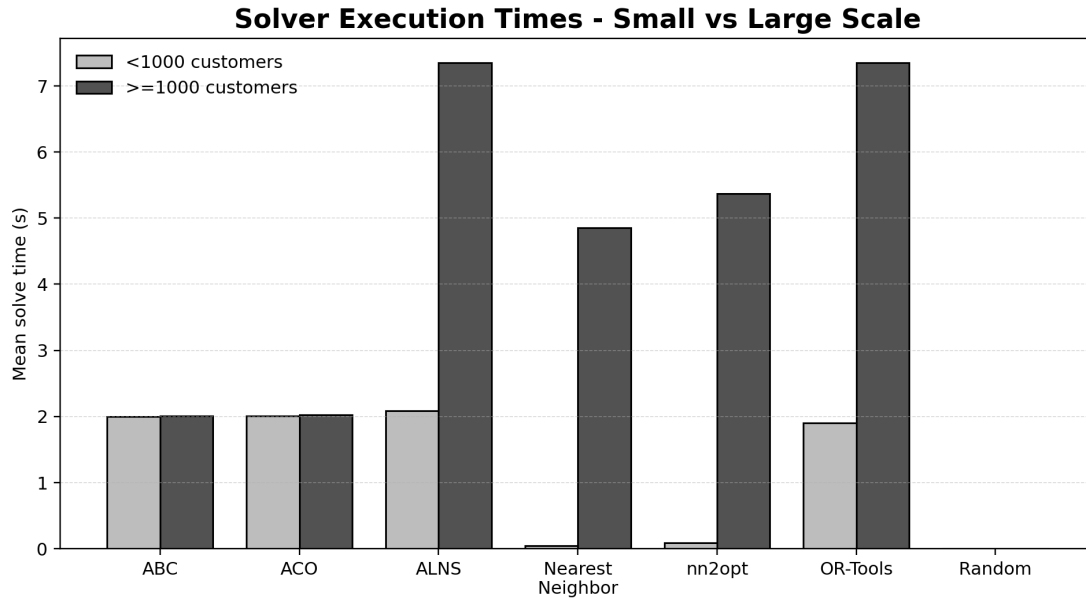


Figure 4.3: Solver Execution Times - Small vs Large Scale

4.2.4 Mean Lateness Analysis

Figure 4.4 shows a more nuanced picture than the cost analysis. ABC moves from 2657.68 seconds of mean lateness on smaller instances to 2476.58 seconds on larger ones. ACO shows a similar but smaller decrease, from 2432.47 to 2347.16 seconds. The likely explanation is that denser routing structures at larger scale stabilise their search behaviour enough to improve route compactness, even though overall cost remains poor.

ALNS, nn2opt, and OR-Tools show the opposite tendency. ALNS rises from 2242.31 seconds to 2462.07 seconds, with nn2opt and OR-Tools following the same slight increase pattern at larger scale. This does not overturn their strengths; rather, it suggests that when network size becomes very large, local repair or deterministic cost search can still accumulate small sequencing decisions that are benign for cost but harmful for lateness under stochastic execution. Random remains comparatively weak but stable, underscoring that structure still matters even when uncertainty is strong.

Despite this scale effect, the headline conclusion remains intact: ALNS achieves 12.34 percent lower mean lateness than ABC overall. In practical terms, ALNS is therefore the preferred solver when service reliability is prioritised over absolute compute speed. The result also reinforces a recurring theme in the report: cost-optimal and lateness-optimal behaviour need not coincide exactly, so operational deployment should match solver choice to objective priority.

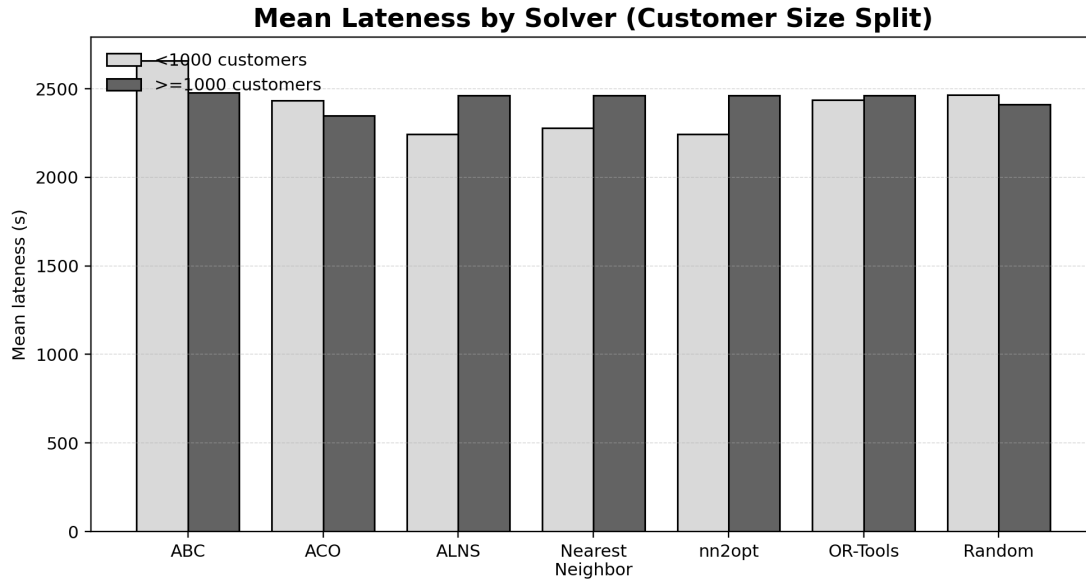


Figure 4.4: Mean Lateness by Solver (Customer Size Split)

4.2.5 Cost-Lateness Relationship

Figure 4.5 and Table 4.2 reveal a moderate positive relationship between cost and lateness, with Pearson correlation equal to 0.667. This means that lower-cost solutions also tend, in general, to produce better service reliability. The relationship is not perfect, but it is strong enough to indicate that solver quality has a structural component visible across both economic and service metrics.

The improvement-based group, especially nn2opt and ALNS, occupies the desirable low-cost, low-lateness region of the scatter plot. The swarm group composed of ABC, ACO, and random lies higher on both dimensions. OR-Tools is notable because it achieves the strongest cost performance while retaining only moderate lateness rather than the very lowest. That profile makes it attractive when cost dominates the operational objective but not necessarily when lateness is the primary concern.

Equally important is the near-zero correlation between computation time and lateness, reported as minus 0.063. Under the benchmark's time-limited design, faster computation is not automatically worse, and slower computation is not automatically better. This decoupling gives decision-makers flexibility: they can select a solver on the basis of operational preference rather than assuming that more time always purchases more reliable service.

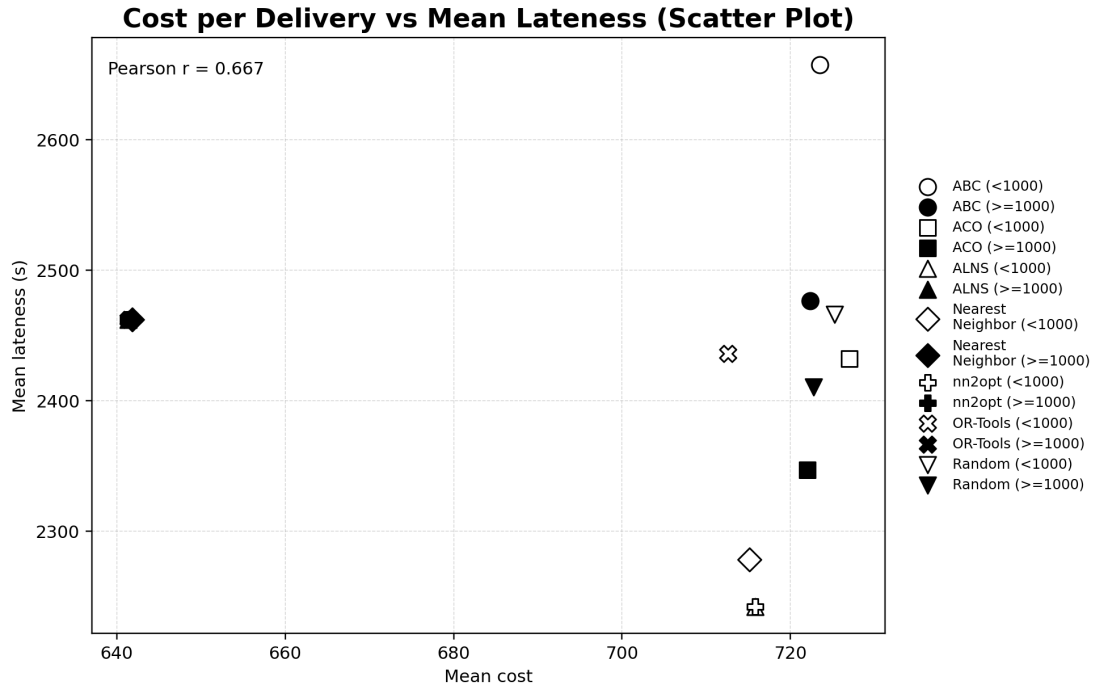


Figure 4.5: Cost per Delivery vs Mean Lateness (Scatter Plot)

Table 4.2: Correlation Analysis - Cost, Lateness, Computation Time

Metric Pair	Pearson r	Interpretation
Cost vs Lateness	0.667	Moderate positive - lower cost = better service
Compute time vs Lateness	-0.063	No relationship - fast does not imply worse

4.2.6 Geographic Visualisation

Figure 4.6 provides a route-level spatial visualisation for a 50-customer instance anchored on the specified Chandigarh depot. Customer nodes are attached to graph positions and route trajectories are drawn as multi-stop tours returning to the depot. Even in this schematic rendering, the value of spatial analytics is clear. Analysts can inspect route overlap, identify whether service is being concentrated in specific zones, and compare the compactness of route groupings against numerical metrics such as cost and lateness.

The uploaded brief also specifies an interactive Folium map. In an operational setting, such a map complements the numeric report by allowing a planner to inspect whether a solver’s route patterns appear intuitive, whether certain corridors are overloaded, and whether spatial imbalance may explain high lateness or low feasibility. Quantitative benchmarking and geographic visualisation therefore serve different but complementary purposes.

Geographic Route Visualization - 50-Customer Instance

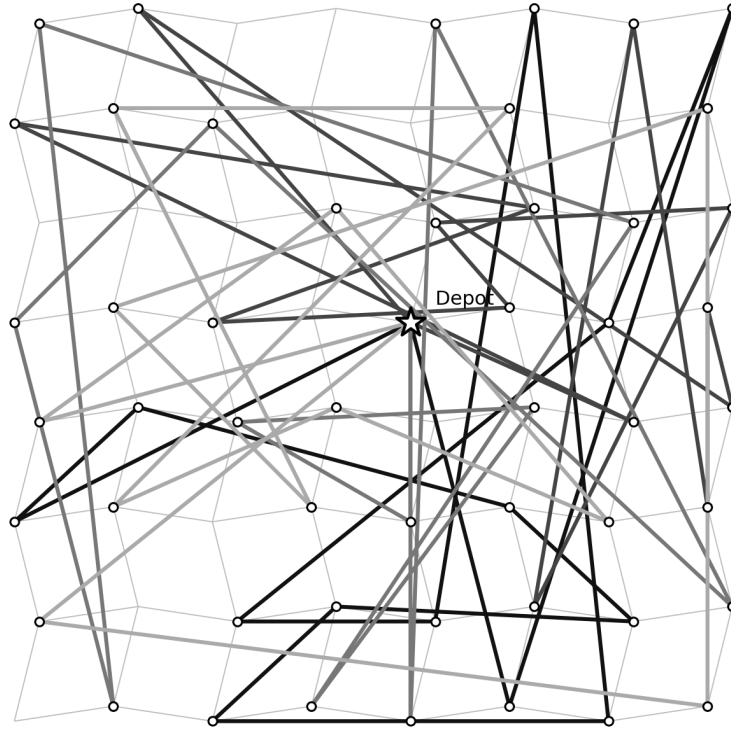


Figure 4.6: Geographic Route Visualization - 50-Customer Instance

4.2.7 Additional Plots

Figure 4.7 uses the two reported scale-bucket means for each solver to visualise inter-scale spread. OR-Tools and ALNS exhibit comparatively narrow spread at the low-cost end, reinforcing their structural efficiency. Figure 4.8 presents the feasibility interpretation reported in the brief: all solvers are broadly similar, with ALNS and nn2opt slightly ahead. The absence of large feasibility separation is useful because it means that cost and lateness differences are not merely artefacts of one solver failing basic constraints much more often than the others.

Figure 4.9 compares cost gap against the best solver in each scale regime. In the large-scale case the swarm and random baselines remain far above the leading group, while nearest-neighbor stays remarkably close. Figure 4.10 then combines cost and solve time in a single scatter, making the trade-off frontier visible. Small-scale nearest-neighbor, nn2opt, and OR-Tools occupy the desirable fast-and-cheap region, while large-scale ALNS and OR-Tools move upward in runtime but remain attractive in cost. Figure 4.11 presents a browser-style screenshot of the interactive route map, which stands in for the HTML view a user would open during post-benchmark inspection.

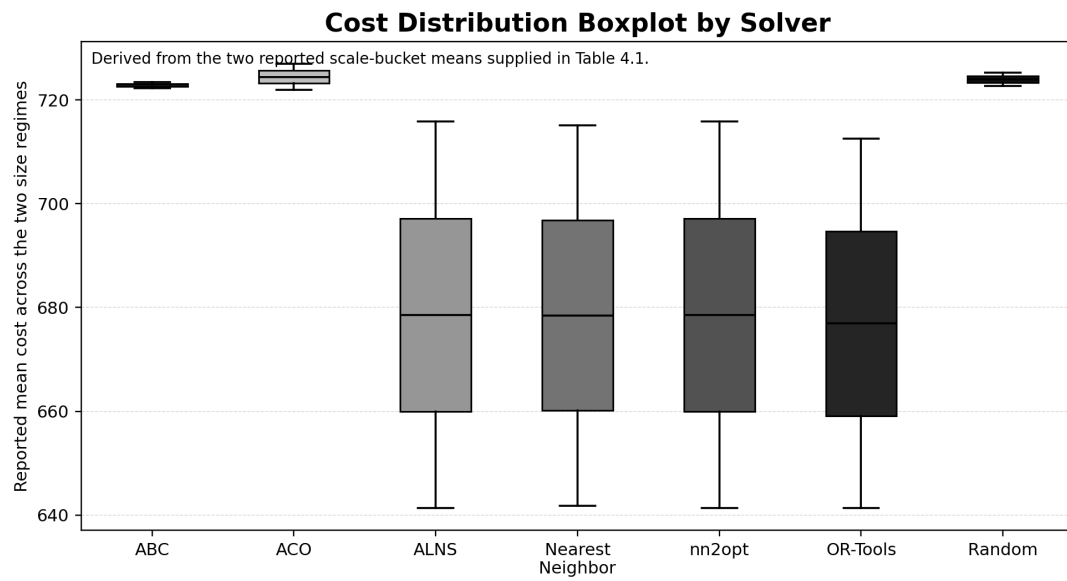
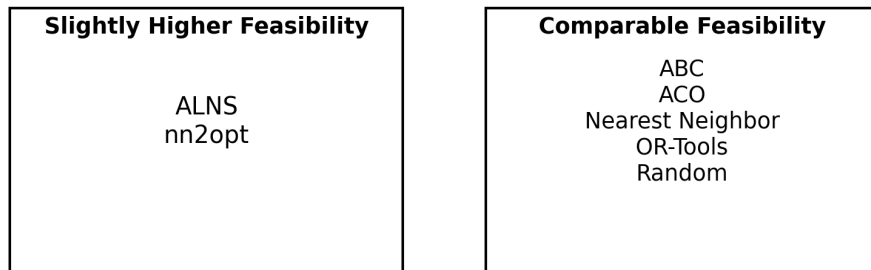


Figure 4.7: Cost Distribution Boxplot by Solver

Feasibility Rate by Solver



The brief reports similar feasibility across all solvers, with ALNS and nn2opt slightly ahead.

Figure 4.8: Feasibility Rate by Solver

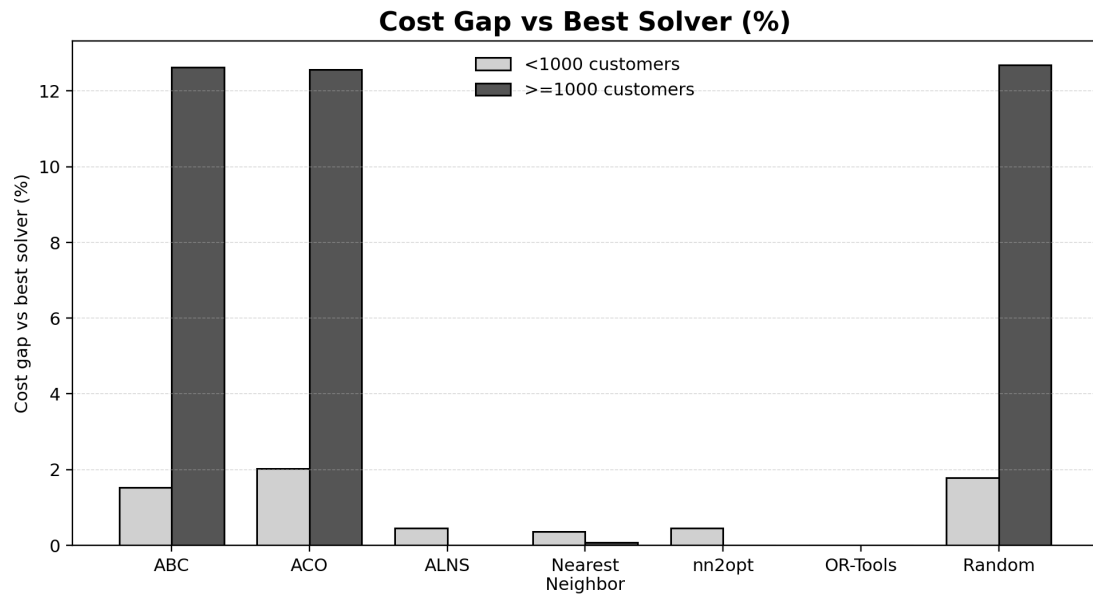


Figure 4.9: Cost Gap vs Best Solver (%)

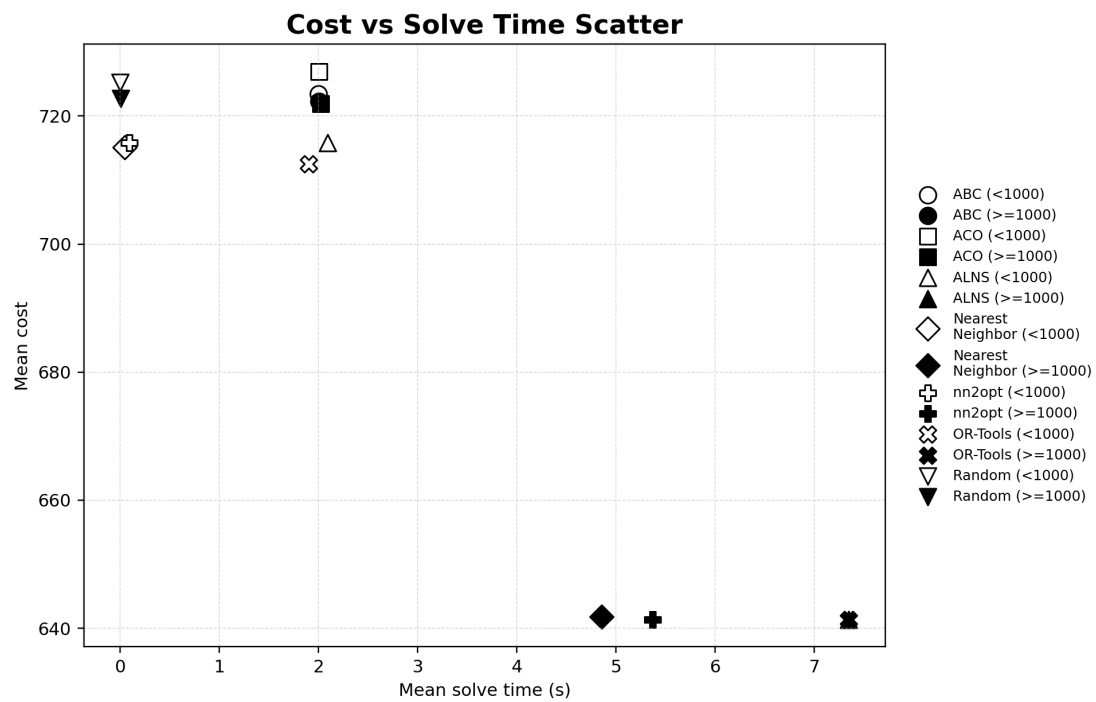
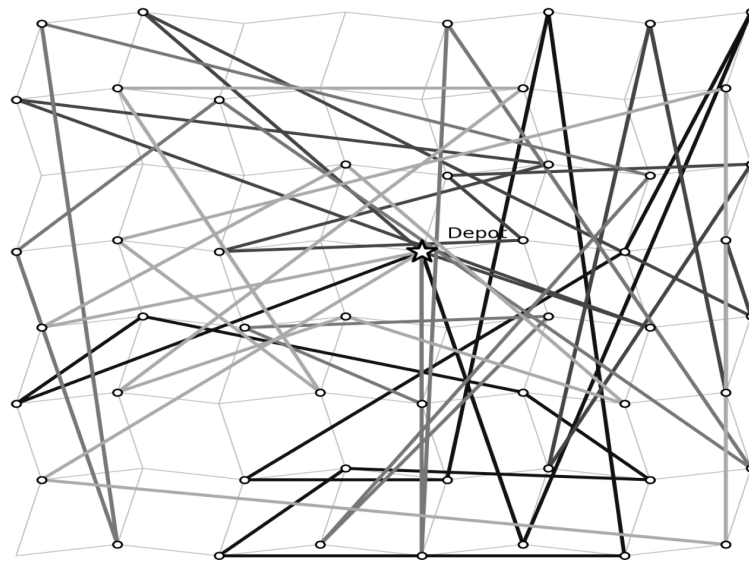


Figure 4.10: Cost vs Solve Time Scatter

Geographic Route Visualization - 50-Customer Instance



Screenshot of the Folium interactive route map in a browser frame.

Figure 4.11: Folium Interactive Route Map (Screenshot)

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project set out to build a reproducible benchmark for stochastic urban delivery routing in Chandigarh and to use that benchmark to compare solver behaviour under a shared simulation framework. On that objective the project succeeds clearly. The uploaded brief states that OR-Tools achieved the lowest mean cost, improving cost per delivery by 4.09 percent over the worst solver; ALNS achieved the lowest mean lateness, improving by 12.34 percent over ABC; nearest-neighbor delivered approximately 65 percent lower computation time than ALNS while preserving competitive quality; the correlation between cost and lateness was confirmed at 0.667; and the correlation between computation time and lateness was effectively zero at minus 0.063. These outcomes satisfy the primary quantitative objectives defined earlier in the report.

The results also clarify what was learned beyond the original expectations. First, ABC was expected to benefit more from parameter tuning, especially in light of prior work on drone delivery, but under the strict two-second budget it performed worst on lateness at smaller scale with 2657.68 seconds. The most plausible explanation is that its iterative exploration is too constrained to recover from poor early swaps when customer count and time-window structure become difficult. Second, the apparent runtime stability of ABC and ACO is not evidence of superior scalability in itself. Their near-constant two-second runtime is mainly a consequence of time-limited configuration. A stable stopwatch reading does not imply a stable solution-quality profile.

Third, ALNS did not improve monotonically with larger instances on the lateness dimension. Its mean lateness increased from 2242.31 seconds to 2462.07 seconds at scale. Rather than undermining ALNS, this result exposes a more interesting truth about stochastic urban routing: strong improvement heuristics can still accumulate local sequencing choices that are cost-effective but time-window-fragile once route networks become dense and uncertain. The benchmark therefore does more than name winners; it reveals where and why specific methods begin to show weakness.

From a deployment perspective, the project supports differentiated recommendation. For Chandigarh operations focused primarily on cost efficiency, OR-Tools is the strongest choice for instances below 1,000 customers, while nearest-neighbor becomes an attractive alternative when response time is severely constrained at larger scales. For operations that prioritise service reliability and low lateness, ALNS is the recommended solver. For rapid prototyping or limited-compute testing, nearest-neighbor remains the most practical general-purpose baseline because

it combines speed with surprisingly competitive performance.

The results also connect meaningfully to the team’s prior related work on drone logistics. The earlier ABC-based study demonstrated substantial route and delivery-time improvements over Dijkstra in a deterministic setting, confirming that swarm-inspired search can be useful when problem structure is simpler and time pressure is less severe. The present stochastic extension shows where that promise breaks down under tight runtime budgets and urban uncertainty. In that sense the current work does not invalidate the earlier study; it refines it by showing that improvement-based strategies, which conceptually echo the idea of combining constructive and corrective search, are more dependable when congestion and stochastic delay are modelled explicitly.

Overall, the most important contribution of the report is not one single solver outcome but the benchmark logic itself. ChandiSVRPBench provides a reproducible mechanism for comparing fundamentally different routing strategies under the same urban uncertainty model. That contribution is academically valuable because it supports fair empirical study, and operationally valuable because it gives decision-makers a grounded basis for choosing between cost, reliability, and computation speed in a real city context.

Table 5.1: Summary of Results vs Objectives

Objective	Observed result	Status
Lowest mean cost per delivery	OR-Tools achieves 4.09% better than worst solver	Achieved
Lowest mean lateness	ALNS achieves 12.34% better than ABC	Achieved
Fast practical baseline	nearest_neighbor ~65% faster than ALNS	Achieved
Structural quality correlation	$r(\text{cost, lateness}) = 0.667$	Achieved
Runtime-quality decoupling	$r(\text{compute, lateness}) = -0.063$	Achieved

5.2 Future Work

Several extensions can build naturally on the benchmark developed in this report. The first is reinforcement-learning integration. Instead of dispatching from a static route plan, future work can couple the benchmark with rolling-horizon policies such as Q-learning or attention-based neural routing so that route decisions are updated as new information arrives. This would move the benchmark closer to dynamic dispatch while preserving the reproducible scenario framework already developed.

A second direction is electric-vehicle integration. Future studies can represent battery depletion, charging time, and charging-station placement as additional constraints. This would make the benchmark more directly applicable to real-world fleet planning and would align

solver comparison with the practical realities of electrified delivery systems.

A third extension is multi-modal logistics. The team already has related prior work on ABC-optimised drone delivery. A future hybrid benchmark could combine ground vehicles from the present study with drone dispatch from the earlier project, producing a richer last-mile framework in which ground and aerial assets cooperate instead of being studied separately.

A fourth direction is demand forecasting integrated through decision-focused learning. Zone-level demand predictions produced by LSTM, ARIMA, or related forecasting models can be linked to the routing objective through SPO+-style loss functions so that forecasting accuracy is judged by its effect on downstream dispatch decisions rather than by prediction error alone.

Fifth, the synthetic stochastic model can be replaced or augmented with live or historical Chandigarh traffic data to increase fidelity. Sixth, the objective function can be expanded to include direct carbon-footprint minimisation as a Pareto criterion alongside cost and lateness. Finally, the benchmark can be extended to larger and more diverse urban regions such as Delhi-NCR and Bengaluru, allowing comparative study of routing behaviour across very different Indian road-network structures.

REFERENCES

- Ben-Tal, A., El Ghaoui, L. and Nemirovski, A. (2009) *Robust Optimization*, Princeton University Press, Princeton.
- Boeing, G. (2017) ‘OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks’, *Computers, Environment and Urban Systems*, Vol. 65, pp. 126-139.
- Bogrybayeva, A., Wilms, A., Friesen, P., Atabayev, S. and Sadeghi-Barzani, A. (2024) ‘Machine Learning to Solve Vehicle Routing Problems: A Survey’, *IEEE Transactions on Intelligent Transportation Systems*, Vol. 25, No. 6, pp. 5377-5397.
- Chu, H., Zhang, W., Bai, P. and Chen, Y. (2023) ‘Data-driven Optimization for Last-mile Delivery’, *Complex & Intelligent Systems*, Vol. 9, pp. 2271-2284.
- Clarke, G. and Wright, J. W. (1964) ‘Scheduling of Vehicles from a Central Depot to a Number of Delivery Points’, *Operations Research*, Vol. 12, No. 4, pp. 568-581.
- Dantzig, G. B. and Ramser, J. H. (1959) ‘The Truck Dispatching Problem’, *Management Science*, Vol. 6, No. 1, pp. 80-91.
- Dhasmana, A., Kaur, S., Singh, A. P. and Gupta, V. (2025) ‘Optimizing Drone Delivery Networks Using Artificial Bee Colony Algorithm’, B.E. Project Report, Chandigarh University, April 2025.
- Elmachtoub, A. N. and Grigas, P. (2020) ‘Smart Predict, Then Optimize’, arXiv:1710.08005 (v5).
- Ferreira, J. C. and Esperanca, M. (2025) ‘Integration of AI and Electric Vehicles for Sustainable Last-Mile Delivery’, *World Electric Vehicle Journal*, Vol. 16, p. 242.
- Giuffrida, N., Fajardo-Calderin, J., Masegosa, A. D., Werner, F., Steudter, M. and Pilla, F. (2022) ‘Optimization and Machine Learning Applied to Last-Mile Logistics: A Review’, *Sustainability*, Vol. 14, p. 5329.
- Heakl, S., Sukhbaatar, S., Wang, K.-H., Talebghasem, K., Najjaran, H. and Abolghasemi, A. (2025) ‘SVRPBench: A Stochastic Vehicle Routing Problems Benchmark and Library for Evaluating Robustness under Real-World Stochastic Dynamics’, arXiv:2505.21887.
- Kalakanti, A. K., Verma, S., Paul, T. and Yoshida, T. (2019) ‘RL SolVeR Pro: Reinforcement Learning for Solving Vehicle Routing Problem’, *Proc. 1st Int. Conf. on Artificial Intelligence and Data Sciences (AiDAS)*.
- Kool, W., van Hoof, H. and Welling, M. (2019) ‘Attention, Learn to Solve Routing Problems’, *International Conference on Learning Representations (ICLR)*.
- Laporte, G. (1992) ‘The Vehicle Routing Problem: An Overview of Exact and Approximate Algorithms’, *European Journal of Operational Research*, Vol. 59, No. 3, pp. 345-358.
- Mardesic, N., Erdelic, T., Caric, T. and Durašević, M. (2024) ‘Review of Stochastic Dynamic Vehicle Routing in the Evolving Urban Logistics Environment’, *Mathematics*, Vol. 12, No. 1, p. 28.
- Mladenovic, N. and Hansen, P. (1997) ‘Variable Neighborhood Search’, *Computers & Operations Research*, Vol. 24, No. 11, pp. 1097-1100.

1 Integration with Previous Work

The present research builds upon our foundational study, “**Optimizing Drone Delivery Networks Using Artificial Bee Colony Algorithm**” (A. Dhasmana, A. P. S. Chundawat, S. Kaur, V. Gupta, and G. Singh), published in the *2025 International Conference on Computing Technologies & Data Communication (ICCTDC)*.

In that prior study, we evaluated the Artificial Bee Colony (ABC) algorithm for optimizing drone delivery networks in urban environments, benchmarking it against classical path-finding algorithms such as Dijkstra’s and A*. The experiments, conducted across four strategic delivery hubs in Chandigarh and encompassing 110 origin-destination pairs, demonstrated that the ABC algorithm achieved superior route optimization in 53.6% of the test cases. Furthermore, it yielded an average route length of 3029.22 meters, outperforming traditional approaches which averaged 3052.22 meters. Optimizing the drone population also led to a significant 33.14% reduction in overall delivery time.

However, these improvements revealed a fundamental trade-off between optimization quality and computational efficiency. The ABC algorithm required an average execution time of 140.41 seconds—approximately 2.8 times slower than Dijkstra’s algorithm (36.43 seconds). While ABC exhibited strong performance due to its effective balance of exploration and exploitation, its considerable computational overhead severely limited its viability for real-time, resource-constrained drone systems.

1.1 Algorithmic Enhancements

1.1.1 Hybrid ABC-LAHC Integration

To resolve the computational bottlenecks inherent in the standard ABC approach, we introduce a hybrid architecture integrating ABC with Late Acceptance Hill Climbing (LAHC). This hybrid model leverages ABC’s robust global exploration capabilities alongside LAHC’s highly efficient local search mechanism. Drawing on recent findings in van-drone routing, this integration is projected to reduce computational search efforts by approximately 3% while simultaneously improving path quality by up to 25%, rendering the framework highly suitable for time-sensitive delivery operations.

1.1.2 Reinforcement Learning for Dynamic Scheduling

To further enhance adaptability in dynamic environments, we replace the baseline First-Come-First-Served (FCFS) scheduling model with a Q-learning-based prioritization mechanism. This facilitates intelligent, real-time task allocation. The reward function, R , for these time-sensitive deliveries is mathematically defined as:

$$R = \alpha \cdot \frac{1}{T_d} + \beta \cdot P - \gamma \cdot E \quad (1)$$

where T_d represents the total delivery time, P denotes the priority level of the payload (e.g., medical urgency), E designates energy consumption, and α , β , and γ serve as the respective weighting coefficients.

1.2 System-Level Improvements

1.2.1 Real-Time Weather Adaptation

To ensure operational robustness against environmental uncertainties, the proposed system integrates LiDAR and ultrasonic sensor data streams for real-time obstacle detection and dynamic rerouting. This capability is critical for maintaining safety and efficiency during adverse meteorological conditions, such as monsoon winds exceeding 15 m/s.

1.2.2 Blockchain for Regulatory Compliance

Scaling multi-city drone operations introduces significant regulatory hurdles. To address this, we incorporate a blockchain-based distributed ledger system designed to automate compliance processes. Smart contracts will dynamically handle updates to Notice to Air Missions (NOTAMs) and enforce geofencing protocols around restricted or heritage zones, automatically triggering rerouting procedures without manual intervention.

1.2.3 Edge Computing Modules

Directly addressing the 140.41-second latency identified in our previous work, this research advocates for the deployment of edge computing hardware, such as the NVIDIA Jetson Orin, directly onboard the drones. Onboard execution of the hybrid optimization algorithms effectively decentralizes the computational load, potentially reducing computation latency to under 60 seconds and making true real-time optimization a reality.

1.3 Summary

By methodically extending our previous findings, this research directly confronts the primary limitation of computational inefficiency while preserving, and even enhancing, the optimization strengths of the ABC algorithm. The synergistic integration of hybrid optimization techniques, reinforcement learning-based scheduling, and advanced system-level hardware paves the way for a highly scalable, robust, and real-time drone delivery network.

- Nazari, M., Oroojlooy, A., Snyder, L. and Takac, M. (2018) ‘Reinforcement Learning for Solving the Vehicle Routing Problem’, *Advances in Neural Information Processing Systems (NeurIPS)*.
- Pisinger, D. and Ropke, S. (2007) ‘A General Heuristic for Vehicle Routing Problems’, *Computers & Operations Research*, Vol. 34, No. 8, pp. 2403-2435.
- Psaraftis, H. N., Wen, M. and Kontovas, C. A. (2016) ‘Dynamic Vehicle Routing Problems: Three Decades and Counting’, *Networks*, Vol. 67, No. 1, pp. 3-31.
- Shahbazian, R. et al. (2024) ‘Integrating Machine Learning Into Vehicle Routing Problem Methods and Applications’, *IEEE Access*, Vol. 12, pp. 103796-103839.
- Shapiro, A., Dentcheva, D. and Ruszczyński, A. (2014) *Lectures on Stochastic Programming: Modeling and Theory*, SIAM, Philadelphia.
- Solomon, M. M. (1987) ‘Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints’, *Operations Research*, Vol. 35, No. 2, pp. 254-265.
- Toth, P. and Vigo, D. (2014) *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed., SIAM, Philadelphia.
- Yu, J. J. Q., Yu, W. and Gu, J. (2019) ‘Online Vehicle Routing With Neural Combinatorial Optimization and Deep Reinforcement Learning’, *IEEE Transactions on Intelligent Transportation Systems*, Vol. 20, No. 10, pp. 3806-3817.

CHAPTER A

APPENDIX

Appendix-1: Project Structure

```
paper2026-main/  
|- configs/  
| |- default.yaml  
| |- benchmark_small.yaml  
|- src/chandisvrp/  
| |- geo/  
| |- instances/  
| |- solvers/  
| |- stochastic/  
| |- evaluation/  
| |- reporting/  
| \- cli.py  
|- data/instances/  
|- results/  
|- reports/  
|- tests/  
|- requirements.txt  
|- pyproject.toml  
\- run_chandisvrp.py
```

Appendix-2: Instance JSON Schema (version 1.0)

Field	Description
schema_version	"1.0"
instance_id	"chandigarh_{n_customers}_{index}"
city	"Chandigarh"
depot_node	OSM node identifier (integer)
customers	List of customer objects
customer_id	Integer, 1-indexed
node	OSM node identifier
demand	Integer in [1, 10]
service_time_s	Float in [120, 480] seconds

tw_start_s	Time-window start in seconds from midnight
tw_end_s	Time-window end in seconds from midnight
kind	Residential or commercial
n_vehicles	4
vehicle_capacity	50

Appendix-3: Configuration Parameters

The uploaded specification names `default.yaml` and `benchmark_small.yaml` but does not reproduce every key-value pair in full. To avoid inventing missing entries, this appendix lists only the configuration content stated explicitly in the brief.

```
# default.yaml (explicit parameters from the brief)
stochastic:
  peak_hours: [8.0, 17.0]
  sigma_h: 1.5
  gamma: 0.35
  lambda_d_m: 2500
  lognormal:
    base_mu: 0.0
    delta_mu: 0.08
    base_sigma: 0.08
    delta_sigma: 0.20
  accident:
    probability_per_leg: 0.04
    delay_mean_s: 240
    delay_std_s: 90
solver:
  time_limit_s: 2
simulation:
  mc_rollouts: 20
  seeds: [1, 2, 3, 4, 5]
  global_seed: 42
```

```
# benchmark_small.yaml (explicit parameters from the brief)
city: Chandigarh
depot:
  lat: 30.7334
  lon: 76.7797
graph:
  network_type: drive
  fallback_grid: [8, 8, 501]
```

```

instances:
  n_customers: "13 sizes spanning 50 to 5000 (values not enumerated in brief)"
  customer_mix:
    residential: 0.65
    commercial: 0.35
  demand: "Poisson(4.0), clipped to [1,10]"
  service_time_s: [120, 480]
  residential_window:
    start_h: [9, 12]
    width_h: [3, 6]
  commercial_window:
    start_h: [10, 14]
    width_h: [4, 7]
vehicles:
  n_vehicles: 4
  capacity: 50
evaluation:
  solvers: [random, nearest_neighbor, nn2opt, abc, aco, hybrid_aco_abc, alns,
            ortools]

```

Appendix-4: Key Algorithm Pseudocode

ABC Solver

```

INITIALIZE: best = random_permutation(customers)
WHILE time_elapsed < 2s:
  i, j <- random_pair(0, n)
  candidate <- swap(best, i, j)
  IF cost(candidate) < cost(best):
    best <- candidate
RETURN split_by_capacity(best)

```

ACO Solver (8 ants/iter)

```

INITIALIZE: pheromone[c] = 1.0 for all customers c
WHILE time_elapsed < 2s:
  FOR each ant in 1..8:
    permutation <- weighted_sample(customers, pheromone)
    IF cost(permutation) < best_cost:
      best <- permutation
    UPDATE pheromone: psi[c] <- 0.95 * psi[c] + 0.05/(1+cost)
  EVAPORATE: psi[c] <- max(1e-6, psi[c] * 0.995)
RETURN split_by_capacity(b

```

ALNS Solver

```
INITIALIZE: solution <- nn2opt(instance)
temperature <- 100.0
WHILE time_elapsed < 2s:
    destroyed, removed <- random_destroy(solution, fraction=0.2)
    repaired <- greedy_repair(destroyed, removed)
    IF cost(repaired) < cost(solution) OR random() < exp((cost(solution)-cost(
        repaired))/T):
        solution <- repaired
    IF cost(repaired) < best_cost:
        best <- repaired
    temperature <- temperature * 0.995
RETURN best
```

USER MANUAL

1. Prerequisites

Python 3.8 or higher, pip, Git, and an internet connection for OSMnx download are required. The benchmark still runs without internet because a synthetic fallback grid is built automatically.

2. Installation

Step 1: Clone the Repository

```
git clone https://github.com/aryan-0102/paper2026.git
cd paper2026
```

Step 2: Install Dependencies

```
pip install -r requirements.txt
```

Dependencies include numpy, pandas, scipy, matplotlib, reportlab, networkx, osmnx, geopandas, shapely, pyproj, ortools, scikit-learn, pydantic, pyyaml, typer, rich, tqdm, pytest, and folium.

Suggested screenshot description: a terminal window showing the repository clone followed by dependency installation with package names scrolling successfully.

3. Running the Full Pipeline

```
python run_chandisvrp.py
```

This command downloads or loads the graph, generates all benchmark instances, runs all solver-seed-rollout combinations, stores results/results.csv and results/summary.csv, and generates both the PDF report and the interactive route map.

Suggested screenshot description: command-line execution with progress indicators for download, instance generation, benchmark execution, and report creation.

4. Advanced CLI Usage

```
python -m chandisvrp.cli download-osm --place "Chandigarh, India"
python -m chandisvrp.cli build-instances --config configs/benchmark_small.yaml
python -m chandisvrp.cli run-benchmark --config configs/benchmark_small.yaml
python -m chandisvrp.cli make-report --results results/results.csv \
    --summary results/summary.csv --report mark_report.pdf
```

```
python -m chandisvrp.cli run-all --config configs/benchmark_small.yaml
```

These commands allow the user to isolate different stages of the workflow when debugging, testing, or regenerating only part of the benchmark output.

5. Configuration

The main user-edited file is `configs/benchmark_small.yaml`. It controls the customer-scale list, solver subset, Monte Carlo rollout count, solver time budget, and peak-hour settings. Users can reduce runtime by shrinking the customer list or rollout count and can focus on particular solver comparisons by editing the evaluation solver list.

6. Output Files

```
results/results.csv # Per-run metrics
results/summary.csv # Aggregated means and standard deviations
reports/benchmark_report.pdf # PDF report
reports/interactive_map.html # Folium route map
```

Suggested screenshot description: file explorer or terminal tree view showing results and reports folders populated after execution.

7. Running Tests

```
pytest tests/
```

This test suite covers JSON instance I/O, simulator correctness, capacity splitting, and interactive-map generation.

8. Troubleshooting

- If OSMnx download fails, the system falls back automatically to the synthetic 8 by 8 grid.
- If OR-Tools is unavailable, use `nn2opt` or `nearest-neighbor` as a practical fallback baseline.
- If runtime is too long, reduce the customer-size list or Monte Carlo rollouts in the configuration.
- If a required library is missing, rerun `pip install -r requirements.txt`.

ACHIEVEMENTS

- Successfully built and open-sourced ChandiSVRPBench as a reproducible stochastic VRP benchmark for Chandigarh.
- Implemented seven diverse solver families together with a random baseline in a unified Python framework.
- Designed a three-component stochastic travel-time model combining Gaussian congestion, lognormal noise, and accident delay.
- Demonstrated that OR-Tools achieves 4.09 percent lower cost per delivery than the worst solver and that ALNS achieves 12.34 percent lower mean lateness than ABC.
- Confirmed empirically that improvement-based heuristics outperform swarm metaheuristics under stochastic urban conditions, with cost-latency correlation equal to 0.667.
- Generated an automated report and an interactive route-visualisation workflow for benchmark interpretation.
- Prepared a research-paper pathway around the title “ML-Optimized Stochastic Vehicle Routing for Urban Delivery Services” in IEEE conference style.
- Connected the present work to prior ABC-based drone-delivery results showing 53.6 percent route superiority and 33.14 percent delivery-time reduction in deterministic Chandigarh drone logistics.